

Served Precision Is Part of the Model: A Quantization Cliff in Proposal Variation, and the Limits of Reproducibility in Agent-Runtime LLM Studies

Anonymous authors

Paper under double-blind review

Abstract

Quantizing a proposer’s weights can leave every metric a discovery loop watches unchanged while collapsing the variation the loop depends on. On a constructive-geometry task at 14B, degrading served quantization leaves viability and validity flat, while at the 2-bit rung the proposer largely stops departing from its parent. An echo bound released before the run that tested it held on five never-sampled seeds — **79% (19/24)** coordinate-verified parent-echo against **6% (1/17)** — and a probe explicitly instructing the model not to copy returned coordinate-identical outputs in **5 of 5** valid rows. The failed proposals are coherent and well-formed, which is why pass/fail instruments report health. The cost is visible only at the loop level: the 2-bit rung takes 1 accepted improvement in 50 calls against 14–16 at the upper rungs, while final best score separates the rungs nowhere. A sixth model family, screen-selected at 31B, reproduces the search collapse under a registered floor-gated test — 0.0 accepted steps against 4.8, with viability intact at both rungs. We then argue that served quantization is one of several serving-path variables an alias-addressed managed runtime leaves unattested, and document a forensic case where an alias drifted within six days — byte-identical prompts returning 30/30 valid against the original 4/30 — and took the question with it. We close with a cheap repair protocol: prompt hashing before sampling, verbatim raw storage, dated alias maps, hash-locked preregistration.

1 Introduction

The same alias, the same prompt text, the same harness, days apart, returned completions one to two orders of magnitude faster than nominally *smaller* models on the same task, and produced valid geometry in 4 of 30 attempts where those smaller models produced it in 30 of 30 and 50 of 60. Nothing in the experimental design predicts either result. Both point to the serving path rather than to anything in the study — an attribution §4 weighs but cannot decide — and the first was visible only because the harness happened to log per-invocation duration.

The weights behind an alias are a promise, not a hash. An alias such as **opus** or **haiku** is resolved at request time by infrastructure the experimenter cannot inspect: different weights on different days, the same weights through a different decode path, or a mixture — none observable from inside the runtime. For most applications this does not matter. For a study whose dependent variable is sensitive to which quantization is served, it matters completely, and the sensitivity is not hypothetical: it was measured directly, on an axis pass/fail metrics cannot see.

This paper makes four contributions.

C1 — the precision cliff, in variation rather than viability. On a value-sensitive constructive task, served-weight quantization measurably moves outputs, but not along the axis the founding hypothesis predicted. Degrading a quantization ladder moves viability and validity by no amount detectable at $n = 50$ per rung — non-rejection, not equivalence, with 7B viability *inverting* — while the 2-bit rung collapses the variation a proposal carries, leaving format and geometry intact. The failure went unreported until coordinates

rather than scores were examined. What carries the claim is registered and replicated: the echo bound $q2_k \geq 60\%$, $q4_k_m \leq 35\%$, written into the runner before execution and held on five never-sampled seeds (19/24 against 1/17), plus a registered must-differ probe that returned its registered branch. Two controls in §3 bound rather than extend the claim: a fixed-parent probe cuts the effect’s magnitude while leaving its direction and 2-bit endpoint intact, and on that probe’s registered centre-only measure the presented lattice is reproduced at 92–98% at every rung — but because the probe’s parents share one lattice, it speaks to radius variation only, not to whether a better-served proposer would move to a different construction. Both waves’ registered decision rules return unusable verdicts (unclassified on wave 1, FAILED on wave 2). Six later registered waves bound generality (§6, §8): a hosted capability ladder does not reproduce the failure mode (0/120, DISSOCIATION), the rung *contrast* transfers to a second geometry while the absolute bands do not, a sixth, screen-selected family clears its upper-rung control floor and reproduces the search-activity collapse under a registered floor-gated secondary (its echo primary underpowered), and the transfers that could not be evaluated — two tasks and five model families below the required competence — are reported under their own registered floor labels.

C2 — a forensic case study. A thirty-invocation arm addressed only by alias, where serving-signature and behavioral evidence together *mildly favour* an unattested serving path without deciding it, and where the favouring evidence — the latency gap — is a working-log observation the released artifact cannot check. §4.1 adds that the favouring is unrecoverable: the alias drifted within six days, so no later call can re-test it.

C3 — a non-identifiability argument, conditioned on the runtime observed. For this agent runtime, the observable set exposed per invocation contains no variable that attests the serving path. We state that set explicitly, show which hypothesis pairs it separates, and name the experiments that would discriminate the two readings behaviorally — two runnable inside the runtime, one requiring a pinned endpoint outside it. We ran both runnable ones (§4.1) under a rule locked beforehand, and the result constrains the argument rather than confirming it: the alias’s behaviour and serving signature both moved within six days, unannounced, so the hypothesis pair the discriminator was built to separate turns out to presuppose a stable referent it does not have. The claim is conditioned on one vendor and one runtime (§8); a runtime exposing a serving-path flag or a weights digest would falsify it for that runtime.

C4 — a repair protocol, including its own failures. The maximal reproducibility such a study *can* achieve — prompt hashing before sampling, verbatim raw storage, dated alias maps, deterministic local scoring, hash-locked preregistration — is specified, implemented, and audited against itself: three of five repairs are fully implemented in the released artifact, and we name the two that are not. The audit extends to the one *instrument* we proposed rather than restated: run against §3’s own rows, on a stack where the serving path is fixed by construction, its firing condition fires anyway, and its obvious repair fails across waves, warm-up and hardware. We withdraw it and report the three mechanisms that broke it.

2 Background and task

The benchmark is circle packing: place exactly N non-overlapping circles inside the unit square (or a $1 \times a$ rectangle) to maximize the sum of radii. It is a standard target for LLM-guided search, and its value here is that candidates are *scored by an exact local evaluator* — no model, no judge, no rubric. A proposal is a list of $[x, y, r]$ triples; validity is a finite set of arithmetic inequalities; the objective is a sum. Every source of run-to-run variance therefore lives on the model side of the interface.

Companion paper. Two prior documents are cited throughout: the *companion paper* is the tier-substitution forensic study released alongside this one; the *antecedent study* is the combined technical report (precision-cliff-paper-combined.md) whose executed quantization results §3 condenses. The companion paper establishes why this task is precision-sensitive rather than merely difficult. Proposals concentrate on constructible templates — a $k \times k$ grid of radius $1/(2k)$, optionally extended with fillers of radius $(\sqrt{2} - 1)/(2k)$ — whose values are exact rationals and algebraic numbers. The template reached for is predicted in closed form by $k^*(N) = \lfloor \sqrt{N} + \frac{1}{2} \rfloor$, the sign of $N - (k^*)^2$ deciding extension versus truncation; across every held-out cell where that rule and the true optimum disagree, the rival optimum was reached in 2 of 34 valid samples. Because predicted values are exact, a proposal either sits on an anchored construction to within rendering precision or it does not, detectable at $1e-6$ — three orders of magnitude below the $\sim 1e-2$

gap between competing constructions. These behaviors are also tier-dependent: one tier truncates templates, another perturbs and mixes radii. A study that cannot attest which weights served it cannot attest which regime it measured.

The instrument, and why the tolerance is registered. The companion’s evaluator scores every row at **two** tolerances, primary declared in advance. Proposers print six to eight decimals, so an eight-decimal tangency misses exactness by roughly $5\text{e-}9$ and a strict $1\text{e-}9$ gate reports a correct construction as invalid on rendering grounds alone. Both tolerances are logged for every row (`valid`, `valid_strict_1e9` in `arm_f_repro.py`), with $1\text{e-}6$ primary because it sits far below the $\sim 1\text{e-}2$ separation between rival constructions. Choosing between them after seeing results would have been the easiest way to fabricate a table, which is why the choice is registered rather than argued, and why §4 reports validity at both.

Several serving-stack variables sit between “the model” as named in a paper and the tokens that arrive: weight quantization and activation precision; fast-mode or speculative decode paths; alias rebinding as new dated ids are released; sampling defaults chosen by the runtime rather than the experimenter; and, in agent runtimes specifically, an inherited system prompt plus user-level instruction files that are not part of the task prompt and are not held fixed across time. Only the last is even nominally under the experimenter’s control, and only if they never edit their own configuration.

3 The precision cliff

This section condenses the executed quantization results of the antecedent study (`precision-cliff-paper-combined.md`: protocol §3.4, results §5.9, summary in contribution 6, scope caveats §6). No new *ladder* data is reported here — every rung figure comes from the antecedent’s runs. One dataset appears here that the antecedent does not analyse: the fixed-parent dispersion probe (§3.5, detail in Appendix B), which controls a confound in the antecedent’s own echo measurement.

3.1 What carries this section

Two registered predictions held, and they are what the claim rests on. The fresh-seed echo bound — `q2_k >= 60%`, `q4_k_m <= 35%`, written into `kaggle_precision_sweep_14b_fresh.py`’s header and pushed before that run executed — was observed at **79% (19/24) against 6% (1/17)** on five seeds never sampled in any prior run. The must-differ decision rule, which specified both branches in advance, returned the copying-is-instruction-insensitive branch at **5 of 5** valid outputs echoing under an explicit prohibition. Around those sit two descriptive replications, labelled as such wherever used: an independently produced imatrix `Q2_K` echoing at 75% against `IQ2_M`’s 43%, and a fixed-parent control preserving direction and the 2-bit endpoint at 92% against 33% in the matched band.

And here is what failed. The fresh wave’s *designated primary* — F1, an improvement-count prediction — was refuted. The dispersion probe’s wave-1 decision rule returns **no category** and wave 2’s returns **FAILED**, its ceiling rung never having run for want of a second GPU. The registered scheme-versus-bit-width control returns its **inconclusive** branch. The 7B ladder refutes the project’s founding hypothesis outright. Everything else — the search-progress statistic, the decomposition, the horizon model, all twenty-nine *p*-values (accounted in Appendix A.4) — is descriptive or post-hoc and carries no claim.

Every 14B figure in this section is recomputed from the raw candidate ledgers, not copied from the antecedent prose; the 7B figures are not. `sec3_ladder_repro.py`, released with this paper, replays each (`quantization`, `seed`) lineage from the coordinate-logged `jsonl` and re-derives the 14B viability, validity, echo, per-seed, must-differ and invalid-row figures below, plus five of this section’s nine Fisher tails ($p = 1.7 \times 10^{-7}$, 3.4×10^{-6} , 0.007, 0.017, 0.056). Ledgers: `sec3_artifacts/**/candidates_precision_14b.jsonl` (200 rows), `..._fresh.jsonl` (100), `..._iq2.jsonl` (100), the two `mustdiffer_*.jsonl` files, each with a `provenance.json` recording SHA-256 and byte length of every served weight file. The remaining four Fisher tails ($p = 5.7 \times 10^{-10}$, 0.001, 0.44, 1.0) and the 7B paragraph are replayed by `sec3_7b_repro.py`. No figure in §3 requires a reader to compute a tail unaided. The $p = 0.001$ figure is a **pooled** contrast — fifteen upper-rung lineages against `Q2_K`’s five, not rung-versus-rung; the paired `Q4_K_M`-versus-`Q2_K` form is $p = 0.048$, and the script prints both.

Inferential status of every p -value in this section. Twenty-nine values fall into four families that must not be pooled: nine Fisher exact tests on counts, six permutation tests belonging to the dispersion probes’ preregistrations, two Spearman orthogonality diagnostics, and twelve post-hoc lineage-level permutation tests on §3.6’s search-progress statistic. **No claim in this paper rests on any of the twenty-nine.** The candidate-level Fisher tests treat rows as independent when they are nested within lineages, overstating their evidence; the six registered permutation tests belong to decision rules that both returned unusable verdicts; the twelve post-hoc tails are unstratified against a seed-crossed design and two sit on the arithmetic floor of their enumeration. Appendix A.4 gives every value, its family, the multiplicity threshold, and the defect each family carries.

Relatedly, “viability is flat” and “validity does not move” report *non-rejection*, not demonstrated equivalence: no equivalence test was run and $n = 50$ per rung is not powered for one (the Wilson interval on $7/50$ is $[0.070, 0.262]$, compatible with a fourfold ratio to the highest rung).

Two notes for a replicator: the running parent advances only on *strict* score improvement, and reimplementing on ties instead undercounts echo by roughly a quarter at the 2-bit rung; and one figure does not reproduce exactly — near-copy fraction among invalid IQ2_M rows is reported as $2/22$, our replay of the stated definition returns $1/22$, a one-row discrepancy at a definitional boundary that affects no claim.

3.2 Ladder and protocol

Proposers: **Qwen2.5-Coder-7B-Instruct** and **Qwen2.5-Coder-14B-Instruct**, official Qwen GGUF releases, served locally through llama.cpp on $2 \times$ NVIDIA T4, SHA-256 of each weight file recorded in provenance. Task: $N = 26$ in the unit square; every lineage starts from the same seeded valid parent — 26 uniform-radius circles on a 6×5 lattice, summed radii 0.89999 (written 0.900). Sampling: temperature 0.8, top-p 0.95, deterministic per-call seed, $\leq 1,200$ new tokens in a 4,096-token context, one bare chat completion per proposal — not agentic, removing the orchestration layer as a confound. Per rung: 5 seeds $\times$ 10 generations (50 loop candidates) plus 6 zero-shot probes. The 7B ladder ran five rungs — FP16 (16.0 effective bits/weight), Q8_0 (8.5), Q4_K_M (4.85), Q3_K_M (3.91), Q2_K (3.35) — for 250 loop rows and 30 probes; the 14B ladder ran the four quantized rungs (FP16 omitted: ~ 29.5 GB does not fit $2 \times$ T4) for 200 loop rows and 24 probes. Every candidate is logged live at evaluation time and scored by the same deterministic evaluator at $1e-6$. The main ladder is a single quantization family — llama.cpp K-quants, never mixed; the sole deliberate departure is the IQ2 control below.

Effective bits per weight are llama.cpp’s nominal figures and *co-vary with the quantization algorithm* rather than indexing it independently — everything in this section is a **Q2_K claim, not yet a bit-width claim**, a distinction the IQ2 control turns from caveat to measurement.

3.3 At 7B the founding hypothesis failed

Viability was flat from FP16 down through Q3_K_M, a $4 \times$ compression: $7/50, 6/50, 9/50, 7/50$, all pairwise Wilson intervals overlapping (FP16 vs Q3_K_M Fisher $p = 1.0$). The 2-bit rung *inverted*, at $16/50$, against the pooled upper four rungs ($16/50$ vs $29/200$, $p = 0.007$ uncorrected, candidate-level) — not FP16 alone ($16/50$ vs $7/50$, $p = 0.056$, ns). Both contrasts are post-hoc. The mechanism is arithmetic: viability reduces almost entirely to count accuracy — of 250 proposals only 45 contained exactly 26 circles and 153 contained 24–25 — the modal count from FP16 through Q3_K_M sits one or two short of the parent shown, while Q2_K’s count distribution is broader (tail to 30–36) and centers on the target. Token truncation is ruled out directly: no 24–25-circle proposal came near the 1,200-token cap (max 687 tokens). The antecedent study names this the **format lottery**. No probe was valid at any rung ($0/30$); the canonical anchored value was never emitted in 280 opportunities; the best score in the whole sweep, 1.79998 (written 1.800), sits 29% below the canonical anchored value, 2.5414. At this scale the proposer sits below the floor at which precision could matter.

3.4 At 14B, a cliff — in proposal variation, not viability

Viability runs 22/50, 22/50, 24/50, 19/50 across Q8_0/Q4_K_M/Q3_K_M/Q2_K, all intervals overlapping, against 87/200 versus 38/200 pooled for the scale contrast (Fisher $p = 1.7 \times 10^{-7}$; the 38/200 denominator is rung-matched, distinct from the 29/200 used above). Validity likewise does not move (18/50, 20/50, 19/50, 18/50); recall stays absent (probes 0/24; best score 1.625, 36% below the 2.5414 anchor). What moved was the capacity to depart from the parent. **Coordinate-verified parent echo** ran 2/18, 3/20, 3/19 across the upper three rungs (8/57 pooled, 14%) against **17/18 (94%) at Q2_K** (Fisher $p = 5.7 \times 10^{-10}$, superseded in magnitude by the fixed-parent control). Every Q2_K seed echoes at or above half its valid rows (7/7, 4/4, 4/4, 1/1, 1/2 — the last is exactly 50%) against at most 2 echoes per seed at any upper rung; 15 of 17 echoes come from three seeds, the remaining two contribute only 1 and 2 valid rows each, so per-seed denominators are too small to support more than the qualitative point. The seed-level improvement contrast (15/15 seeds improving above the boundary vs 1/5 below, $p = 0.001$) is post-hoc and single-run; the candidate-level echo contrast is descriptive only.

Data loss, and a coordinate-verified re-execution. The original 14B run’s hosting session expired before the output directory was retrieved, destroying per-candidate jsonl, probe texts and checkpoints; only the verbatim live console log survived, which never recorded coordinates. The rung was re-executed end to end against the same SHA-256-pinned weights, seeds, prompts and sampling parameters, with three logging-only additions: per-candidate coordinates, redundant console echo, and quantitative predictions embedded before execution — the runners are released (`sec3_artifacts/runners/`). This is a **re-execution, not a replication**: llama.cpp’s seeded sampling is deterministic, so console-logged predictions were guaranteed to hold and carry no evidential weight. The falsifiable predictions were the coordinate-level ones — whether score-matches were verbatim copies or same-sum rearrangements. Both held (Q2_K echo $\geq 60\%$: observed 94%; each upper rung $\leq 35\%$: observed 11–16%), and the check *refuted* the original score-inferred estimates on the upper rungs, reclassifying 4 of 12 presumed echoes as rearrangements, making the verified cliff sharper (14%→94%) than the inferred one (21%→94%). A public Kaggle kernel, **precision-redetermin** (owner handle withheld for double-blind review), re-ran 112 of the 224 generations (Q4_K_M and Q2_K) on the original hardware class, and **all 112 reproduced their archived per-row digests exactly** (verdict SUPPORTED). Two caveats travel with it: the original llama-cpp-python build is unrecoverable (wheel version-pinned 0.3.34), and two earlier executions of the same kernel drew a P100 and correctly returned VOID_ARCH for every row instead of comparing across architectures — cross-hardware nondeterminism is Yuan et al.’s finding (§7), not a defect.

Fresh seeds replicate the echo cliff, and a registered prediction fails. The protocol was run on five never-before-sampled seeds (2222, 3333, 5555, 7777, 9999) at the two decisive rungs, 100 new coordinate-logged candidates. Echo replicates: **19/24 (79%) at Q2_K against 1/17 (6%) at Q4_K_M** (Fisher $p = 3.4 \times 10^{-6}$, candidate-level, descriptive), every fresh Q2_K seed again majority-echo (4/5, 4/5, 5/6, 2/2, 4/6). **The wave’s designated primary was not the echo bound but the improvement count, and it failed**: the runner header labels F1 “Improvement (seed-level, primary)”; F2, the echo bound, carries no such label. F1 forecast $\leq 2/5$ Q2_K seeds improving past baseline; 3/5 did, against 5/5 at Q4_K_M (Fisher $p = 0.44$). The antecedent study demotes the improvement collapse to a fragile correlate: the **echo rate** is the cliff’s replicable signature. The $p = 0.001$ seed-level figure quoted earlier in this section is the superseded one.

Mechanism: the cliff survives an explicit prohibition. A registered must-differ probe held the parent fixed and demanded the output not be identical, ≥ 3 circles changed. At Q2_K the model returned a coordinate-identical copy in **5 of 5** valid outputs; at Q4_K_M, 1 of 5, with all four non-echo outputs scoring above the parent. Under the pre-registered rule ($\geq 50\%$ echo $\rightarrow$ instruction-insensitive; $\leq 20\% \rightarrow$ instruction-sensitive; inclusive both bounds; the rule’s subject is Q2_K alone, Q4_K_M’s 1/5 reported without a branch) Q2_K falls in the instruction-insensitive branch at the maximum possible rate, while format adherence stays intact.

Scheme-graded, not bit-graded: the IQ2 control. A registered control ran two independently produced 2-bit-class quantizations from a third-party imatrix-calibrated repository, SHA-256-pinned: **imatrix Q2_K** (~ 3.4 b/w) and **IQ2_M** (~ 2.7 b/w — fewer bits, different scheme family, same calibration). Echo: 24/32

(75%) for imatrix Q2_K against 12/28 (43%) for IQ2_M ($p = 0.017$), must-differ echoes 6/8 versus 0/2, seeds improving 3/5 versus 5/5. The cliff replicates on an independently produced Q2_K at essentially the official file’s matched-seed rate (75% vs 79%). The registered bit-width question returns its **inconclusive branch** — 43% falls between the thresholds ($\geq 60\%$ generalizes, $\leq 35\%$ attributes to algorithm) — and neither is claimed. The direction points away from raw bit width: 6% (Q4_K_M) $\rightarrow$ 43% (IQ2_M, ~ 2.7 b/w) $\rightarrow$ 75–79% (imatrix and official Q2_K). Read at face value this says a lower-bit quantization from a different family preserves more proposal variation than a higher-bit K-quant, but the fixed-parent control below withdraws that reading (§3.5).

Closing the selection-effect loophole. Echo rates are computed among *valid* outputs, and a verbatim copy is valid by construction, so a high echo rate is in principle compatible with a model attempting many mutations that all break. Classifying every non-valid row rules this out: there is no garbage — every failed 14B output at every rung is a fully parsed, malformed circle list, never truncation or non-coordinate text. At official Q2_K the *invalid* rows are themselves majority near-copies (18/32 re-execution, 15/26 fresh seeds; typically 26 of 27 circles unchanged). Counted over the whole output distribution, official Q2_K emits copies or near-copies in roughly 70% of attempts. The scheme gradient reappears in the failures — near-copy fraction among invalid rows: 0/33 (fresh Q4_K_M) $\rightarrow$ 2/22 (IQ2_M) $\rightarrow$ 5/18 (imatrix Q2_K) $\rightarrow$ 15/26 and 18/32 (official Q2_K). A validity-filter explanation predicts mutation-dominated invalid rows; the invalid rows are copy-dominated.

3.5 A fixed-parent control narrows the contrast

The echo rates above are measured against each lineage’s *running* parent, and those parents are not distributed alike across rungs: Q2_K lineages rarely improve, so their parent mostly remains the seeded 0.900 grid, while upper rungs climb away from it. Parent quality is confounded with rung, in the direction that flatters the result.

A fixed-parent dispersion probe removes that confound: it runs the same Qwen2.5-Coder-14B at the same three rungs (Q8_0 was registered but never attempted on either wave — the 8-bit file does not fit the single P100 allocated; both `provenance.json` files record `"q8_0": {"skipped": "needs 2 gpus, have 1"}` — so the probe covers three of the ladder’s four rungs), holding the parent constant at each of six preset scores from 0.88 to 1.65, logging coordinates and an echo flag per row across two independently seeded waves (v1: 288 rows, 165 valid; v2: 432 rows on out-of-sample salted seeds). Both waves carry their own preregistration (Appendix B); **wave 2’s own registered verdict is FAILED**, because its addendum requires a ceiling rung (q8_0) that never landed, so every wave-2 descriptive is exploratory.

The registered centres-only echo measure shows **no cliff** (92–98% at every rung, both waves), but all six of the probe’s “parents” share identical centres, differing only in radii — the probe shows the model one lattice throughout, so this measures whether that lattice is copied, not whether the model would move to a different construction. On the axis the design *can* measure — radius variation — the cliff appears: within the fixed lattice, upper rungs perturb radii in 31–51% of valid rows with no stable ordering between them (reversing between waves), while at Q2_K almost every valid row is an outright copy, leaving 2/49 (v1) and 5/84 (v2) that perturb radii. Full decomposition, both waves, and the registered analysis table ($JT = 4460.0$ against null mean 4498.5, $p = 0.686$ on the primary echo measure; NED $p = 0.030$; rarefaction $p = 0.953$; quality fork $p = 0.454$) are in **Appendix B**.

Restricted to the 0.88–0.90 band the loop actually occupies, the loop-vs-fixed-parent contrast is **6/18 versus 11/12 — 33% against 92%**, not the loop’s 15% against 94%. Appendix B works out three consequences. The cliff’s direction and 2-bit endpoint survive, but its spread does not: the loop’s 14%-to-94% figure is partly an artifact of where each rung’s running parent sits, and the fresh-seed 79%-vs-6% figure and the invalid-row near-copy gradient carry the same confound. The IQ2 scheme-gradient ladder (§3.4) is weakened, since if Q4_K_M’s matched rate is 33–52% rather than 6%, IQ2_M’s 43% may not be separated from it at all — so we withdraw the gradient as evidence about bit width and keep it only as a loop-condition description. And the registered upper-rung bound “ $\leq 35\%$ ” held under the loop condition (11–16%) but would have **failed** under fixed parents (52–61%): it was registered against the loop design and is correctly scored there, but is not a general property of those rungs. The probe is not a substitute for the loop measurement, since a fixed

parent removes the lineage history, but it is the better-controlled comparison on the one axis where the loop is confounded.

3.6 What the loop actually did: search progress at the lineage level

Every measure above is a property of a single call. This subsection reports what the *loop* did with those calls — post-hoc, appearing in no preregistration, released as `sec3_search_progress.py`. Define an **accepted improvement** as a valid output scoring strictly above its lineage’s running best. On the registered 14B ladder, across five lineages of ten generations each: Q2_K takes 0, 0, 0, 0, 1 accepted steps (**1/50** total; final bests 0.900, 0.900, 0.900, 0.900, 1.625); Q3_K_M 4, 3, 4, 2, 2 (**15/50**); Q4_K_M 6, 3, 2, 3, 2 (**16/50**); Q8_0 2, 2, 5, 1, 4 (**14/50**). Four of five Q2_K lineages take zero steps in ten generations. Because an echo can never be an accepted improvement (`accepted <= valid - echo` by construction, with gaps 0, 1, 1, 2 across the four rungs), most of this contrast is the echo contrast in complement — the table’s addition is the *unit*: a count of steps over a lineage, the quantity a loop’s cost is denominated in.

The fresh wave’s designated primary was a prediction about this quantity, and it failed. F1 (“Improvement, seed-level, primary”): $\leq 2/5$ Q2_K seeds and $\geq 4/5$ Q4_K_M seeds improving. Observed 3/5 vs 5/5, $p = 0.44$ — **F1 failed**. The echo bound F2 carries no primary label. The finer, post-hoc form of the same rows — steps rather than whether-improved — gives 3 versus 14, $p = 0.0079$, and does separate the rungs; that gap is analytic flexibility, not a repair of F1, and no claim rests on it. The most this paper claims for the fresh wave: the echo bound held on never-sampled seeds, and its designated primary did not.

Quantization gates the frequency of departure; whether it also degrades departure quality is open. What the 2-bit rung loses is the occasion to depart: 10 departures in 133 valid fixed-parent outputs against Q4_K_M’s 74 in 164. Whether those rare departures are also *worse* ones is not settled here — ten departures against a self-set floor of twenty-five is underpowered, and the wave designed to supply more departures returned UNINFORMATIVE by its own control floor (§8) — so the question stays open.

This points at a repair and complicates it: if frequency is gated, forcing departure should recover the search — but the must-differ probe did exactly that and returned 5/5 echoes anyway. Whatever gates departure at 2 bits was, on this probe’s evidence, not reachable by instruction.

Replication is partial, and the level of the test decides it. On an exact lineage-level permutation test (distinct from the dispersion probes’ JT tests), the registered ladder gives $p = 0.0008$ against the pooled upper three (15,504 splits) and $p = 0.0079$ against Q4_K_M alone (252 splits, the floor of a five-versus-five enumeration). The fresh seeds replicate: 3 steps against 14, $p = 0.0079$ (also floor-bound). **The IQ2 control does not:** imatrix Q2_K takes 6 steps against IQ2_M’s 16, exact tail $p = 0.135$. A call-level Fisher test on the same counts returns $p = 0.028$; we report the lineage-level figure instead because calls within a lineage are not exchangeable, which costs us the IQ2 replication but avoids banking an unsupported significance.

The outcome metric is blind to all of it. Final best score after ten generations does not separate the rungs anywhere: registered ladder Q2_K mean 1.045 vs pooled 1.115 ($p = 0.593$) and vs Q4_K_M’s 1.054 ($p = 0.937$); fresh seeds Q2_K *higher*, 1.153 vs 0.999 ($p = 0.421$); IQ2 control 1.087 vs 0.922 ($p = 0.318$). This is non-rejection at five lineages per cell, not equivalence — the score support is lumpy and discrete (0.900, 0.936, 1.040, 1.300, 1.625), so one lucky jump in a Q2_K lineage lands on the same rung that four small Q4_K_M steps climb to. The finding is the *blindness*: an order-of-magnitude collapse in search activity, 1 step against 14–16, leaves the reported outcome statistically indistinguishable at this horizon.

Whether the gap opens at longer horizons is registered here as a **prediction, not a result** — we did not run it, and Appendix C reports the horizon-power exercise we did run, including a withdrawn power figure and the structural reason final best score is the wrong dependent variable at any horizon.

At 7B the statistic does not replicate, and reverses. No coordinates were stored at 7B, but accepted improvements are computable. Step counts: FP16 6/50, Q8_0 2/50, Q4_K_M 3/50, Q3_K_M 4/50, **Q2_K 7/50** — the 2-bit rung takes the *most* steps of five, permutation tail $p = 0.116$ against pooled others, $p = 0.238$ against Q4_K_M. Final best score does not separate the rungs at this scale either ($p = 0.158$ against pooled others, 0.119 against Q4_K_M — completing the six outcome-level tails, $p = 0.12$ –0.94 across ladders, behind the abstract’s statement that final best score separates the rungs nowhere; Appendix C).

Validity at 7B runs 4–12 of 50 at every rung, so no rung is searching enough for the statistic to have room to separate; the reversal sits with the viability inversion §3.3 already reports rather than contradicting it. What this establishes: **the search-step collapse is a 14B observation, not a property of the quantization ladder as such.**

What varied, and what did not. Across every run in this section the inference stack was *constant within each run* — same llama.cpp build, sampling parameters, harness, and one fixed hardware allocation (2× T4 for all four ladders, a single P100 for both dispersion-probe waves, per each `provenance.json`’s `gpus` field). Ladder timings and probe timings are therefore never compared to each other. The only thing that changed *inside* a run was which SHA-256-pinned weight file was loaded — this section measures the effect of **served-weight quantization**, not a decode path, kernel, batching regime, or speculative-decoding scheme. That degrading this one serving-stack element selectively collapses proposal variation does not establish that other serving-stack variables do the same; that generalization is the hypothesis §4 finds an unattestable instance of, not a result established here.

The transfer §3 needs is narrow: a degraded *served quantization* can leave every pass/fail axis intact while removing constructive competence. Measured at $N = 26$ on locally served open weights, one scale pair, one quantization family plus one control — not at §4’s cells, not inside the agent runtime (§8).

4 Forensic case study: the opus_alias arm

Setup. The study’s third tier was requested as a specific dated top-tier model, but the agent runtime accepts only an alias — no dated identifiers — so the request was unsatisfiable, and *undetectably* so: nothing in the runtime’s response surface reports which weights served a call. We label the arm `opus_alias` throughout. Thirty invocations, ten each at three held-out cells ($N = 13, 21, 31$), predictions registered in `arm_o_preregistration.txt` before any sampling, digest 211718c6b58d627f17e34aa73ad6142b89c7f39048e5e19a8cc864d63c281738.

Serving signature. Completions returned in **2.8–9 s** across all thirty invocations. On the same harness and task the Haiku tier returned in **75–250 s** and Sonnet in **150–1170 s** — one to two orders of magnitude slower for nominally smaller models, holding uniformly across all thirty invocations. This is a working-log observation (STATE.md §8, §8b), **not** captured in the released artifact: `arm_f_raw.json` rows carry exactly four keys (`arm`, `n`, `raw`, `sample_id`), no duration field, and `arm_f_repro.py` contains no timing capture at all. A reader can recompute every validity figure and cannot check a single latency figure — the one item of §6’s disclosure standard this paper does not itself implement (§4.1’s arms close the gap prospectively, not for these thirty rows). A second apparent anomaly (uniform reported-token counts, $\sim 49.9k$ across all thirty) is withdrawn: the harness logs a single aggregate usage figure over a large fixed system prompt and a task prompt uniform in length (520 characters) across cells, so near-uniformity is exactly what the design predicts, and the residual spread is plausibly tokenizer-level variation between “13”, “21” and “31”. No contemporaneous control exists: the three tiers were not interleaved, and load demonstrably varied within the study (five invocations elsewhere were rejected by a concurrency cap before reaching a model, §6 item 3), so load cannot be excluded as a latency contributor. Speed is not precision on the one stack we fully control — across §3’s ladders throughput does not order by bit-width (14B Q8_0 slowest of four rungs at 14.9 tok/s against Q2_K’s 22.5; 7B FP16 slower than every quantized rung; §6 `sec6_cv_canary_audit.py`). Those gaps are tens of percent against this arm’s one-to-two orders, so the magnitudes are not comparable. But speed and precision are demonstrably separate axes even on a fixed stack, and throughput is not a portable fingerprint — it is not stable across hardware either (§6 item 4).

Behavioral signature — recomputable, both tolerances. Validity was **4/30 (13%)** at 1e-6 and **4/30** at strict 1e-9 (per cell 3/10, 1/10, 0/10) — tolerance-invariant collapse. Comparators, recomputed from the released file: Sonnet **30/30 at 1e-6**, **27/30 at 1e-9**; Haiku (bare) **50/60 at 1e-6**, **47/60 at 1e-9**. A fourth comparator, present in the released file and not previously reported: 70 `trace` rows at the same three cells (20, 30, 20), belonging to the companion paper’s trace-elicitation arm, score **63/70 valid (90%)**, per cell 18/20, 28/30, 17/20 — against `opus_alias`’s 4/30, Fisher $p = 1.1 \times 10^{-13}$; pooling all three comparators, 143/160 against 4/30, $p = 1.0 \times 10^{-16}$ (comparators do not differ detectably from each other: `trace` vs `bare` $p = 0.30$, vs Sonnet $p = 0.099$). Its prompt is substantially different from the bare prompt while the

serving path is the same one the other local arms used, so a large prompt change produces nothing resembling the collapse — a control §4 otherwise lacks. Two caveats: this arm is a reuse from the companion paper’s question, and `arm_f_raw.json` does not distinguish that paper’s pilot rows from its `trace_v2` rows, which it forbids pooling — the 63/70 above therefore pools rows the companion paper forbids pooling, because the released file provides no separating field. No claim in §4 rests on it; the 4/30 against 30/30 and 50/60 contrast stands without it. (A 45-row prior Haiku slice that once served as comparator was overwritten in place by a later 100-invocation wave with colliding `sample_ids` and no batch/date field, and cannot be reconstructed — the concrete case motivating §6 item 3.)

The failures are geometric, not formatting: all 26 invalid rows overlap (2 additionally contain zero-radius circles, but both still overlap once those are removed — gate ordering, not disjoint failure modes; one, $N = 21$ sample 5, has 12 of 21 radii at zero). The errors sit far outside rounding noise (one $N = 31$ row: a deficit of 0.06 against tolerance $1e-6$, $\sim 6 \times 10^4$ times over). The attempted constructions shifted family and are *more* ambitious than the grid-plus-filler templates the other tiers produce: at $N = 13$, all ten rows place four $r = 0.25$ corner circles plus filler radii in an Apollonius-like arrangement (classifier returns **structure: other** for all ten); at $N = 21$, mixed-radius grids with fillers; at $N = 31$, a coarse $r \approx 1/6$ grid with border strips. The four valid samples score below the registered trap value by very different margins — at $N = 13$, 1.2646/1.2873/1.4142 against a 1.625 trap (13–22% below); at $N = 21$, 2.07 against 2.1 (1.4% below, effectively at it). Registered scorecard: P-O1 (on-trap rate $\leq$ Sonnet’s 1/30) **held** at **0/30** — nearest miss 2.07 against 2.1, off by 0.03, fifteen times the registered $2e-3$ window; P-O3 (multi-radii fraction ≥ 0.9) **held**, 4/4; P-O2 (rival-argmax) and P-O4 (best valid score ≥ 2.2588835) **failed** outright (0 rival-argmax against Sonnet’s 6/30; best 2.07 against the required value) — two held, two failed, none unevaluable.

Two hypotheses. H1, *serving-path degradation*: a fast decode path erodes the arithmetic precision needed to close a tangency while leaving construction choice intact — precisely §3’s signature. H2, *genuine tier property*: the resolved weights really do attempt harder constructions and execute them less reliably. **What the observable set can and cannot separate.** The runtime exposes, per invocation, exactly four things: completion text, wall-clock duration, an aggregate usage figure, error codes (call this set O). On text, both hypotheses induce the same distribution. On timing they are not symmetric: H1 predicts the latency observation directly; H2 needs an auxiliary — “a fast path was in use but had no behavioral effect” — which is H1’s mechanism minus its causal claim, strictly more complex. The observations mildly favour H1 within this snapshot, but cannot *decide*, because the auxiliary is not itself testable through O : no element reports which decode path served a call. **The irreducible residue is the unattestability of the serving path, not a blanket impossibility of behavioral discrimination.**

A pinned dated endpoint, outside the agent runtime, would be informative in both directions but cannot attest what the *alias* resolved to on the day sampled — the counterfactual is gone. We did not run this experiment; its absence is a limitation of this paper. Two further routes were named but not run in earlier drafts and have now been run (§4.1): **prompt variation** (hand the alias a fixed, known-valid packing and ask it to verify/repair — decoupling execution from ambition), and **a dated re-snapshot** (re-sampling the same alias later, informative either way, without ever attesting either snapshot’s serving path).

4.1 Both follow-ups, run

Preregistered together in `arm_r_preregistration.md`, committed before the first invocation, run on 2026-08-07 through the same alias. Ninety invocations: thirty re-issuing the byte-identical bare prompts (**arm B2**), sixty on the repair task (**arm R**), split across two cells, a three-step precision ladder, and two tiers. All figures replay from `arm_r_analysis.py`.

A repair score without an anchor would be uninterpretable — arm B2 is the anchor, registered as P-R0 at $\leq 12/30$ **valid**, disconfirming branch written out in advance.

Arm B2 disconfirmed P-R0 at the widest margin the design allows: $N = 13$, $3/10 \rightarrow 10/10$ (best valid 1.776141); $N = 21$, $1/10 \rightarrow 10/10$ (2.276800); $N = 31$, $0/10 \rightarrow 10/10$ (2.794180); pooled $4/30 \rightarrow 30/30$, Fisher $p = 7.8 \times 10^{-13}$. The p -value rejects the hypothesis these are draws from one success rate; it is **not** a test that the serving path changed (one path per condition — pseudoreplication with respect to that question).

Wall clock and usage were logged as two further channels, and they diverge. Wall clock went from 2.8–9 s to **68.7–594.1 s** (median 228.2), overlapping the comparator range it was once one to two orders below, but this comparison is uncalibrated in both directions (eyeballed log ranges vs a runtime-reported field, no contemporaneous 2026-08-07 comparator, no timing claim registered) and is consistent with, not evidence for, the validity change. The usage channel fails entirely, and instructively: reported usage went from uniform ~49.9k to 58k–195k, a 3.35-fold spread that looks like a second signature, until arm R’s same-day, same-alias, different-task rows show usage uniform to **1.01-fold (54,377–54,803)**. The spread tracks the task, not the date (within B2, usage and duration correlate at $r = +0.69$). **The usage observation is withdrawn again, here.**

What the surviving result does not do: retract any 2026-08-01 observation. **What it does:** take the section’s question away rather than answer it. H1 and H2 remain well-formed about the 2026-08-01 call; what is gone is any route to testing them, because the only handle — the alias — no longer addresses that call, and no observable reports when it changed. A later call is a sample of an unknown condition, not a second sample of the same one. Fast-and-13% then slow-and-100% is what H1 predicts if only the decode path moved, and equally what H2 predicts if the weights themselves changed; two conditions, no randomization, and a simultaneous difference in date, load, decode path and possibly identity distinguish none of these.

Arm R ran anyway, because it was registered. Six cells — $N \in \{13, 31\} \times \delta \in \{1e-2, 1e-3, 1e-4\}$ — five invocations each at alias and Sonnet, on a grid-plus-interstitial packing exactly tangent everywhere with one circle slid by δ . Results: **59/60 exact** (all 5/5 except alias $N = 31, \delta = 1e-3$ at **4/5**), recall 1.00 in every cell of every tier, the single miss a near-miss false positive (true clearance 7.089e-04), zero empty responses anywhere, detection flat in δ across two orders of magnitude at both tiers. Of seven registered predictions, three held, four failed: P-R1 (exact ≥ 0.6 at $\delta = 1e-2$) held; P-R5 (Sonnet $\geq$ alias at $\delta = 1e-2$) held; **P-R4 held vacuously** (predicted empty responses would not become rarer — there were zero anywhere); P-R2 (“monotone non-increasing,” satisfied by flat detection) scored FAILED on one cell only ($N = 31$ ’s 1.0/0.8/1.0, the single false positive making the sequence rise); P-R3 ($\geq 2/5$ drop $\delta = 1e-2 \rightarrow 1e-4$) and P-R6 (Sonnet beats alias at $\delta = 1e-4$) both failed outright — P-R6 became structurally unreachable once Sonnet hit ceiling.

The H1 signature the probe was built to detect was not detected — weaker than absent, and it matters at this size. Five invocations per cell put the one-sided 95% lower bound of a 5/5 cell at 0.549; the design excludes a precision effect of roughly 45 points or more and says nothing about a smaller one. **And we may not read it:** P-R0’s failure triggers the registration’s fourth branch — arm R measures a serving path that cannot be tied to the 2026-08-01 one, so no H1/H2 conclusion is drawn from it, even though arm R’s own result is clean and would otherwise read as evidence against H1. Two things arm R does establish on the path it actually measured: repair-style probes are not where this path fails on this substrate ($\delta = 1e-4$ at $N = 31$, four orders below coordinate scale, found by 5/5 both tiers), and there is no tier contrast left to measure here, since both tiers sit at ceiling. Registered predictions P-O2/P-O4, refuted on 2026-08-01, are not rehabilitated by 2026-08-07’s numbers — they were registered about a path no longer addressable. Arms B2 and R capture per-row duration and usage, closing §6’s disclosure gap prospectively (not for the original thirty rows).

5 What is repairable and what is not

The repairs below are implemented in `arm_f_repro.py` and companions, not merely proposed, and where they are not, this section says so.

Implemented. Prompt text is pinned and SHA-256 hashed before any sampling, one hash per problem size: $N = 13$ 32db485bea625ff9f39f4723ebf1a01f337559a9e2cf567fb486928f71f7f8df, $N = 21$ a415425b4ed5a57ea9b6f09c2328508f12370e1624734e1c5ed32913741795a9, $N = 31$ a664d003cbf1c0eca51bae5b3a1d072071eb34756725a7491d6a2e8fa3b78e92. (Each prompt is 520 characters, template plus two-digit N . `arm_f_prompts.json` carries five cells — $N = 13, 17, 31, 35, 37$ — verifying the $N = 13$ and $N = 31$ digests directly; $N = 21$ is absent from that file but checkable, since substituting “21” for “13” in the released $N = 13$ prompt reproduces a415425b... byte-exactly.) Every raw output is stored verbatim, parsed or not. Scoring, validity and construction classification are deterministic, local, computed at both tolerances per

row. Predictions with explicit falsifiers and a disconfirmation rule are written before the first invocation and externally timestamped; §4 publishes the full registration digest.

Implemented partially, and named. Run date is recorded with the alias→dated-id mapping in force that date, but the released map contains a single entry (for the Haiku proposer), and `RUN_DATE` is a single hard-coded date that is *not* the `opus_alias` arm’s actual run date. The paper’s headline provenance repair is unimplemented for precisely the arm the paper is about. Raw storage has a matching defect: rows carry no batch, wave or run-date field, and `sample_id` collides across arms and waves, so the released file cannot be sliced back into the arms as originally reported.

Not repairable from inside. Sampling parameters (temperature, top-p, top-k) are not exposed and are the parameters that shift the output distribution most. The alias→weights binding is a promise, not a hash, reproducible at any time; past runs cannot be re-executed against the weights that produced them. The subagent inherits a system prompt and user-level instruction files, not part of the task prompt and not held fixed across time. The serving path is not attested at all — no flag distinguishes fast-mode from standard decode, the exact variable §4 needed and could not obtain.

Does output-text fingerprinting close the gap? Wimbauer et al. (2026) show serving-stack differences are detectable from output text alone, but three things keep it from settling §4 as run. Fingerprinting is *classification against reference signatures*, and none exist for this vendor’s closed serving paths. A tempting-but-wrong objection is that the arm’s thirty completions (mean 578 characters) are too thin: these methods issue *fresh* probes against a live alias, so the relevant budget is queries one is willing to spend, not stored text (Leshin et al., 2026, fingerprint from fixed-prompt output distributions and detect changes of inference stack and quantization, and Bruckner, 2026, recovers model identity from roughly a hundred single-token queries; both are affordable). What survives is only retrospective: fresh probes cannot be sent to the path that answered our thirty calls in the past, because it is no longer addressable. Most fundamentally, a positive fingerprint would identify a *stack*, not attest *weights* — it could strengthen H1’s antecedent without touching the auxiliary H2 needs. Fingerprinting is the most promising published route to narrowing this class of question, is not runnable on this corpus, and a study banking reference signatures while the endpoint is available would be a genuine advance over what we did.

Table 1: Repair protocol: which observability gaps are repairable from inside the runtime, by what mechanism, and what residual risk survives the repair.

Item	Repairable?	Mechanism	Residual risk
Prompt text	Yes	SHA-256 pre-sampling, digests published	None
Raw outputs	Partial	Verbatim storage, failures included	No batch key; <code>sample_id</code> collides across waves
Scoring / classification	Yes	Deterministic local evaluator, both tolerances	None
Predictions	Yes	Hash-locked prereg, externally timestamped	Registration errors — disclose, don’t amend
Model identity	Partial	Alias + run date + dated-id map in force	Map covers Haiku arm only; <code>RUN_DATE</code> wrong for <code>opus_alias</code> ; alias may be repointed
Serving-signature stats	Partial	Absent from §4’s arm, implemented in §4.1’s	§6 item 4’s firing condition tested against §3’s own rows and withdrawn; disclosure half retained
Sampling parameters	No	—	Unknown distribution shift
Alias → weights binding	No	—	Silent substitution; past runs unrepeatability

Item	Repairable?	Mechanism	Residual risk
Serving path (fast-mode)	No	—	§4-class confound; fingerprinting not runnable on this corpus
Inherited sys-tem/user prompt	No	—	Unlogged context drift

The scorer is model-written, and that is a circularity worth naming. This paper’s integrity argument rests on deterministic local scoring, but that code was written with assistance from models in the same family as the runtime §4 studies. The available answer is reimplementing: `sec3_ladder_repro.py` was written independently, from the definitions in the text, and re-derives §3’s 14B figures and five of nine Fisher tails; `sec3_7b_repro.py` replays the rest; §4 was re-scored from `arm_f_raw.json` by a separately written geometric checker. Agreement is exact except the one invalid-row near-copy count noted in §3.1. This is not independence in the strong sense — the same author directed both implementations — but it establishes that figures follow from the released rows under stated definitions rather than a particular scorer’s quirks.

6 Implications

Every LLM-evolution, best-of- N and iterative-refinement study run through a managed agent harness inherits this list — FunSearch-style program-evolution loops, self-refinement pipelines, tier-ladder comparisons naming “the model” by alias. Such studies are not thereby wrong; they are unrepeatable in a specific way — demonstrated here for one vendor’s serving path and one runtime (§8) — and §3 establishes that served weight precision is **one** unrepeatable variable the outcome is sensitive to. Whether others behave alike is a hypothesis raised, not tested. The correct response is disclosure, not retraction.

The capability alternative, tested and bounded (wave 4, registered). The strongest alternative reading of §3 — echo is what any weak proposer does, so a weak model rather than a 2-bit rung would suffice — was left standing because no prior experiment varied capability independent of precision. Wave 4 did (`wave4_prereg_tiers_heilbronn.md`, SHA-256 7b3e67dc..., published before sampling): three hosted capability tiers ran the identical Heilbronn $n = 13$ lineage protocol, same seed parent, same echo definition, tools forbidden (`tool_uses` 0/120). **Result: 0/120 coordinate-verified echoes — every tier at 0% against a registered $\leq 30\%$ bound — verdict DISSOCIATION** (roughly a 3% bound at this n , not an exclusion at any rate). The registered secondary (echo monotonicity across tiers) returned **NOT EVALUATED** (its 10-echo floor unmet). No proposal in any tier improved on the seed (0/120; wave 3’s GPU ladder later went 0/495 on the same task and parent — substrate-independent stall). The sharper finding: **the tiers fail differently, not less**. Stuck GGUF rungs echo at 46.9–63.1% (§8), stuck hosted tiers at 0% (though on LABS, a third task, stall-echo rates *reversed*, §8 — Heilbronn-specific, not a law of stalls). The registered template check finds valid outputs scoring exactly zero (three-plus collinear points) at 52.5% for the weakest tier, 27.5% for the middle, and **97.5% for the strongest**, whose 39/40 zero-scoring outputs are family-similar symmetric lattices, pairwise distinct at 6 dp. The 2-bit rung’s signature failure is *copying the parent shown*; the hosted tiers’ signature failure is *ignoring the parent and emitting a template*. A capability ladder does not reproduce the quantization ladder’s failure mode even where it fails more often. Scope limits are the prereg’s own: served precision, sampling parameters and serving path per tier are unattested (C3); one date; one harness wrapper; and the dissociation bounds the capability confound over the tiers sampled only.

A minimum disclosure standard — mostly an endorsement, with one addition. GUIDE-LLM (Feuerriegel et al., *Nature Human Behaviour* 10:1182–1186, 2026), a 14-item consensus checklist from an 80-expert panel, already mandates exact model version and access method, prompts and system instructions, sampling parameters, validation; parallel software-engineering guidance exists (Baltes et al., 2026; Korn et al., 2026), and Siddiq et al. (2025) show artifact badges across 640 papers do not guarantee reproducibility. Items 1–3 below restate requirements those documents already impose, with agent-runtime specifics.

Item 4 is the addition, and the artifact that motivated §4 does not implement it — §4’s central evidence is a working-log observation rather than a checkable file, while §4.1’s arms implement it and the

value of doing so is visible: the serving-signature change those arms document is checkable in a way §4’s is not.

1. **Alias, run date, and the alias $\rightarrow$ dated-id map in force on that date.** Two lines of the harness header; pins nothing but converts an unfalsifiable claim into a dated, checkable one. Implemented here for one proposer only — a map omitting the arm under study is not provenance (§5).
2. **Prompt hashes computed before sampling, and published in the paper** — a digest kept in a private file is a check, not a disclosure (§5 publishes ours).
3. **Verbatim raw outputs, including failures, with a batch key.** In our own logs, five invocations were rejected by a concurrency cap before reaching a model in the Haiku wave; scoring them as proposer failures would have moved that arm’s reported validity from 32/45 (71.1%) to 32/50 (64.0%) — 7.1 points absolute, 10.0% relative (this worked example uses the superseded 45-row slice, no longer reconstructible after an in-place overwrite — the overwrite is itself an instance of the defect this item exists to prevent). The batch key is the part we got wrong.
4. **Serving-signature statistics, with a firing condition.** Per-invocation duration and usage, logged and reported as distributions with an explicit canary condition: flag an arm when duration CV falls below one third of a neighbouring tier’s on the same cells, or median duration differs by more than an order of magnitude under a same-window interleaved schedule; disaggregate usage before reporting spread.

We then ran item 4 against our own data, and it has a false-positive mode. §3’s ledgers carry per-invocation `wall_s` on a *fixed* inference stack (fixed llama.cpp build, single P100, same sampling parameters, same harness), so whatever the canary reports there is not a serving-path change (`sec6_cv_canary_audit.py`). **It fires:** duration CV among valid rows is 0.0603 (Q4_K_M) vs **0.0143** (Q2_K), ratio 0.237, below the one-third threshold; wave 2 replicates, 0.0692 vs **0.0200**, ratio 0.289 (permutation $p < 0.0001$ both waves); Q3_K_M does not fire on either wave (ratios 0.949, 1.066) — the canary is selectively wrong at exactly the rung §3 is about. **The mechanism is output degeneracy, not serving:** splitting rows by echo status, duration CV is low among echo rows at every rung (Q4_K_M 0.0210 on $n = 34$, Q3_K_M 0.0141 on $n = 31$, Q2_K 0.0113 on $n = 46$) and high among non-echo rows at every rung (0.0783 on $n = 31$, 0.0778 on $n = 20$, 0.0294 on $n = 3$) — an echo re-emits its parent verbatim, lands on the same token count, takes nearly the same time. Q2_K’s aggregate CV is low because 46 of 49 valid rows are echoes, a property of the output, not the serving path. **Item 4 needs a control it did not carry:** report duration dispersion conditioned on output identity, or exclude repeated outputs before computing CV. This does not overturn §4, whose evidence is an absolute one-to-two-orders-of-magnitude latency gap on a non-echoing arm (4/30 valid), but it is a competing explanation §4 never named for the class of evidence it relies on.

The repair fails, in a way that matters more than the repair would have. Removing the confound — throughput over non-echo valid rows only — looked disjoint on wave 2 (Q4_K_M [17.911, 18.126], Q3_K_M [15.731, 15.938], Q2_K [16.596, 16.718] tok/s). Three things break it. First, **wave 1 is not disjoint:** its Q4_K_M range [16.455, 18.188] entirely contains Q2_K’s [16.703, 16.753], and an earlier draft quoted only medians, hiding the overlap. Second, **a cold-start effect silently did the filtering:** the first invocation against each parent runs at $\sim 92\%$ of steady-state throughput uniformly across all twelve rung $\times$ wave cells (0.918–0.929), and on wave 2 every cold row happens to be an echo or invalid, so the echo filter removed them by accident, making “non-echo” silently a **warm** set — a verifier’s fresh probe is always cold, so it would read the 92% band, not the published one. Third, **the band does not survive a hardware change, and the order inverts:** the same SHA-256-pinned Q2_K file runs 16.62 tok/s on the P100 and 22.48 on 2 $\times$ T4 (Q4_K_M: 17.94 and 21.45); on the P100 Q4_K_M is faster, on the T4s Q2_K is faster, not a shifted scale but an inversion. Throughput is also length-dependent (within-rung correlation between completion length and tok/s ≈ -0.9), which fixes none of the three. These three mechanisms are the transferable part for anyone attempting timing-based attestation: dispersion is confounded by output identity,

warm-up gradients can masquerade as between-condition effects and vanish by accident under an unrelated filter, and a per-file timing band inverts across hardware — structural, and the case where the verifier does not control the hardware is exactly the attestation case. What this licenses about C3, stated narrowly: it does not show no observable attests the served file (output-side methods exist, §7), only that **the one instrument class a verifier can compute from what an agent runtime already returns — timing — failed every variant we could construct**, worsening when the verifier loses the hardware. What survives: throughput does not order by bit-width on either ladder (14B: Q8_0 slowest at 14.91 tok/s against Q2_K’s 22.48; 7B: FP16 slower than every quantized rung), robust to matching token counts — relevant to §4’s inference from “unusually fast” to “cheaper serving path,” though the magnitudes (tens of percent here, one-to-two orders there) are not comparable. **Item 4 is therefore withdrawn in its dispersion form and not replaced**, keeping only the disclosure half (log duration and disaggregated usage, report distributions).

5. **A parent-echo canary, runnable in under an hour.** The only *instrument* still proposed, and it inherits item 4’s failure class stated up front: its firing condition is confounded by a variable the verifier cannot observe — here, parent quality, not output degeneracy — but unlike item 4 the confound is one a practitioner *can* control by choosing the parent. **The screen:** hold a parent configuration fixed, make it weak (your loop’s seed configuration), issue 25–50 proposal calls at production settings, log emitted structure verbatim, compute the fraction of valid outputs whose structure equals the parent’s (order-insensitive, six decimals). **A rate $\geq 60\%$ is a red flag; $\leq 35\%$ is normal.** **The five-call version:** append an explicit prohibition (must differ, ≥ 3 elements change, unchanged counts as failure) and issue five calls — 5/5 echoes is the signal, as observed at Q2_K against 1/5 at Q4_K_M. These thresholds are the bounds preregistered in the fresh-seed runner (§3.4), which held at 79% against 6%. **Why the parent must be weak:** §3’s probe shows echo depends heavily on parent quality — pooled over both waves, at parent 0.880 Q4_K_M is 7/30 (23%), Q3_K_M 8/26 (31%), Q2_K 19/21 (90%); at 1.650, Q4_K_M is 15/24 (62%), Q3_K_M 14/20 (70%), Q2_K 19/23 (83%) — against a good parent a healthy Q4_K_M proposer echoes at 62–74% and Q3_K_M at 70–78%, past the red line. Only at the weak end is separation clean (23–42% healthy against 88–90% degenerate; one healthy cell at 42% exceeds the 35% line, so an AMBIGUOUS reading does not imply degradation). Not a calibrated precision detector — §3’s own scheme control returns inconclusive — but supported as a tripwire: something is wrong with what is serving this proposer. Released as `echo_screen.py`.

What vendors could expose. A weights digest per completion would make the alias→weights binding attestable. A sampling-parameter echo would make an unset temperature auditable. A serving-path flag — one boolean for fast/speculative decode versus standard — would have separated §4’s two hypotheses outright.

7 Related work

Pineau et al. (2021) document the NeurIPS reproducibility program and code-submission checklist now carried across ICML, ICLR and AAAI — the canonical anchor for reproducibility-in-ML. A separate line shows fixed models on fixed hardware do not return identical outputs under varying batch composition: Yuan et al. (2025) report accuracy swings up to 9% on a 7B reasoning model under bf16 as batch size, GPU count and version change, with He et al. (Thinking Machines Lab, 2025) locating the cause in the absence of batch-invariance in matmul, RMSNorm and attention kernels and shipping batch-invariant replacements. Our contribution against that background is the *addressing mode*: batch-invariance work assumes the stack can be named and controlled; the alias interface makes the relevant variable unaddressable rather than merely uncontrolled.

Chen et al. (2023) is the canonical precedent for **behavioral change behind a stable identifier** — the same GPT-3.5/GPT-4 identifiers returned substantially different behavior across snapshots months apart, establishing that a stable alias does not denote a stable model. Our contribution against it is not that aliases drift but what the drift does to a *specific measured capability* inside a selection loop, on an axis invisible to

the pass/fail instruments such a loop already runs. Leshin et al. (2026) state our hazard more completely: an endpoint can remain “healthy” while effective model identity changes through weights, tokenizers, quantization, inference engines, kernels, caching, routing or hardware; their Stability Monitor fingerprints an endpoint from fixed-prompt output distributions with a summed energy-distance statistic and permutation tests, detecting changes of model family, version, inference stack *and quantization*. This obliges §5’s explicit narrowing of C3: such methods **detect change** between snapshots; none **attests which decode path served a particular past call** — a much smaller claim than “the question is closed.” Bruckner (2026) fingerprints model identity from roughly a hundred single-token queries — the cheapest available change detector, though it addresses identity rather than quantization.

The attestation side of C3 is crowded, and our position is confirmatory. Cai et al. (2025) audit model substitution in commercial APIs, show software-only detection unreliable against exactly the substitution class that includes quantized serving, and arrive at trusted execution environments as the only route to provable integrity; Zhang et al. (2026) extend the audit to gateway routing dilution; Schnabl et al. (2025) run verifiable benchmarks inside enclaves. §6’s item 4 (serving-signature disclosure, standing in for the decode-path attestation this line shows software cannot provide) restates this line’s conclusion as a reporting obligation; §§3–4 add a measured demonstration, inside a selection loop, of how much the unattested variable can matter.

Two 2026 quantization studies bracket §3’s result. Rababah et al. (2026) introduce *correctness agreement* and show behavioral divergence persisting while accuracy and perplexity stay stable — §3’s thesis in generic form; our addition is the loop setting and a coordinate-exact echo metric. Zhou et al. (2026) separate two 2-bit failure modes — signal degradation (training-free repairable) and computation collapse (not), the latter manifesting as incoherent, stop-word output. Our Q2_K failure is the opposite surface signature at the same nominal rung: fully coherent, format-valid, geometry-valid output that happens to be a copy — a failure mode passing every legibility check is the harder one to notice.

Copying is not a phenomenon we discovered: ShinkaEvolve (Lange et al., 2025) ships two-tier novelty rejection sampling precisely because LLM proposals are routinely near-duplicates of their parent. What we claim is narrower: the *rate* of it rises sharply at the 2-bit rung of one ladder on one task at one scale — 14% to 94% in the loop condition, discounted to 33% against 92% once parent quality is controlled — so a mitigation tuned at one precision may be asked to do substantially more work at another with no signal that the amount changed.

Output variation has its own measurement tradition — distinct-n (Li et al., 2016), Self-BLEU (Zhu et al., 2018), the Vendi score (Friedman & Dieng, 2023) — and evolutionary computation made variation a first-class objective long before LLM proposers existed: novelty search (Lehman & Stanley, 2011), MAP-Elites (Mouret & Clune, 2015), and FunSearch’s island model (Romera-Paredes et al., 2024), which engineers against proposal-diversity decay operationally. None of these instruments is run as a serving-path health check, and none has been connected to served precision as the independent variable. Parent-echo is deliberately narrower than a corpus diversity score: it names the mechanism — reproduction of the conditioning input — rather than the symptom of samples resembling one another, which is what lets the must-differ probe separate cannot-depart from narrow-sampling.

The LLM-evolution literature catalogues its own failure modes and does not contain ours: PACEvolve (Yan et al., 2026) names context pollution, sampling mode collapse and weak inter-generation collaboration, all at full precision and addressed with orchestration machinery; a degraded-proposer failure in which the model silently returns its parent appears nowhere, consistent with our reading that it is a substrate property rather than a scaffold property. Lin (2026) document a rise-and-collapse failure in self-training loops via reward over-optimization — an unrelated mechanism but the nearest precedent for a loop degrading through a variable its own metrics do not surface.

The quantization-effects literature measures degradation on static single-shot benchmarks. Marchisio et al. (2024) find automatic metrics systematically *understate* quantization harm (1.7% average drop on Japanese by automatic metrics against 16.0% under human evaluation) — §3’s thesis from a different domain. GPTQ (Frantar et al., 2023) and AWQ (Lin et al., 2024) are cited as method references, with an explicit family caveat: §3 sweeps llama.cpp K-quants and one i-quant, a different scheme family, and neither GPTQ, AWQ

nor bitsandbytes was tested. Four 2026 entries sharpen the gap without closing it: Kurt (2026) evaluates the same llama.cpp ladder on the same Llama-3.1-8B-Instruct our wave 7 adopts, one-shot throughout, the closest published instrument to ours; the “Quantization Meets Reasoning” line (Li et al., 2025; 2026) and Lv et al. (2026) establish reasoning-shaped capability is disproportionately fragile under compression — the static-benchmark shadow of the effect measured dynamically here; Chang et al. (2025) explain which inputs break low-bit models via residual-stream magnitudes; Mix-Quant (Lu et al., 2026), the only published treatment of quantization for *agentic* inference, finds decode phase rather than prefill is precision-sensitive, convergent with a proposal loop failing at generation time. None places a quantized proposer inside an iterative search and measures novelty.

Madaan et al. (2024) measure seed variance across 280 models on 13 benchmarks; Miller (2024) supplies power/uncertainty machinery for reported deltas. Both matter here specifically because §5’s residue is orthogonal to both: a serving-path change is not a draw from a noise distribution being sampled, it is a change in which distribution is sampled, invisible to within-run error bars. Wimbauer et al. (2026) on output-text fingerprinting is engaged in §5, where it bears on identifiability rather than serving as background.

For benchmark lineage, arXiv 2605.29268 studies the same 26-circle objective via LLM-guided program synthesis with an explicit best-of- N comparator, complementing our companion paper’s attractor account. Reported values saturate — AlphaEvolve at 2.635, later systems clustering at 2.635983283 (ShinkaEvolve), 2.63598308 (HELIX, 2603.07642), 2.636 (GigaEvo 2511.17592, AdaEvolve 2602.20133), with further unverified reports (SeaEvo 2604.24372, ThetaEvolve 2511.23473) — a field agreeing to the eighth decimal while addressing models by alias is reporting agreement it cannot attest.

Zhou et al. (Nature, 2024) report larger, more instructable models attempt more and err more in QA — our observation is a constructive-regime instantiation of the same shape, cited as corroboration. Hash-locked, externally-timestamped registration of LLM experiments is an emerging standard (HindsightBench 2607.18867, concurrent; 2607.07184; 2606.27687; 2606.11217); we claim only the combination of hash-locked registration of *exact closed-form point predictions* evaluated in a held-out container.

8 Limitations

We observed a single vendor and a single agent runtime; whether other runtimes expose more (dated identifier, sampling echo, serving-path flag) is untested, and C3 is stated conditionally for that reason. The ladder itself covers two model scales, 7B and 14B, a bound set by the free-tier GPU allocation (32 GB across $2 \times T4$ — a 70B q4_k_m file alone exceeds it); whether the cliff exists at larger scales is untested by design, not oversight.

The serving-signature evidence is circumstantial *by construction* and unshipped: its evidence class is a working session log, not the released artifact, the arms were not interleaved, so load is not excluded as a latency contributor. We infer a serving path from a latency distribution because no direct observation is available; a reader who declines the inference is left with the second hypothesis, equally unattestable, and C3 stands either way. The token-count observation is withdrawn as a non-finding (§4), illustrating why §6 item 4 demands disaggregated usage.

The `opus_alias` arm is $n = 30$ from one batch window. Both follow-ups this paper named — the repair probe and a second serving-signature snapshot — have now been run (§4.1) and converted an inference into a residue rather than an observation: the snapshot showed behaviour, latency and usage spread all moving together within six days, so the hypothesis pair §4 poses cannot be settled by any experiment addressed through that alias, including the discriminator built for it. **§4’s H1/H2 question is not open pending more data; it is unaskable in the form §4 asks it.** Any study whose treatment is a vendor alias inherits this hazard.

The fixed-parent dispersion probe, on which §3’s strongest control rests, carries four limits (detailed in Appendix B): it presents one lattice, so it measures radius variation and cannot observe a proposer moving to a different construction (centre-echo 92–98% at every rung is a statement about the only lattice shown); neither wave yields a usable registered verdict (wave 1 no category, wave 2 FAILED for want of the ceiling rung); the registered spread statistic is reconstructed rather than replayed, its generating kernel absent from the corpus; and the decomposition carrying the §3 result is post-hoc, with only its 2-bit endpoint replicating

across waves (the upper-rung ordering reverses). What survives all four: within the presented lattice, radius perturbation collapses at the 2-bit rung, on two independently seeded waves.

The loop-level search-progress statistic (§3.6, detail in Appendix C) carries three limits: it is post-hoc and a finer form of a registered prediction (F1) that failed; most of it is the echo result in complement; and it does not replicate at 7B or on the IQ2 scheme control (lineage-level tail $p = 0.135$). What survives is one ladder at one scale with a fresh-seed replication that does not clear a Bonferroni threshold across its own family, and an outcome-level null reported as non-rejection at five lineages per cell. The accompanying conditional-quality analysis is worse: it rests on ten 2-bit departures against a floor of twenty-five we set ourselves, in an unlocked note, the day the analysis became computable — it confirms nothing, and its exclusion holds by 0.8 points against the least favourable comparator, reversing entirely if one of ten departures flips. It is also a fixed-parent measurement, and whether departure quality holds up inside the loop (which includes lineage history the fixed parent removes) is not established by it. This is the thinnest load-bearing thing in the paper.

A rival reading we do not screen: recall rather than construction. A sceptic can read the template-anchoring story as pretraining contamination — recall of packings seen rather than construction — and nothing here rules that out; the cells the anchoring argument leans on ($N = 13, 21, 26, 31, 35$) are exactly the population most likely to appear in a training corpus, giving the rival reading its best case. A three-probe contamination screen is specified (`arm_canary_contamination_audit.py`) and **not run** — its first probe measures instruction-following rather than contamination (a sampling-time marker bears on training-corpus contamination only if the marker was in the training corpus), its second passes a memorised template plus one filler, its third trips on any packing with non-lattice fillers (the same false-positive mode that undid §6 item 4). All three defects are recorded in the file’s own header; a contamination screen for this task is open work.

Six later waves address generality, each reported in its own registration’s labels. (Waves are numbered by design label, not execution order; no design labeled wave 6 was ever registered or sampled, and the wave-8 label survives only in a seed-calibration pilot repurposed for wave 3 — every registered wave is reported here.) Wave 3 (`wave3_prereg_heilbronn.md`, Heilbronn $n = 13$, chosen because a regular grid scores exactly zero on it — separating template-emission from parent-copying, a distinction circle packing cannot make since its seeded parent *is* the template; pushed as a public Kaggle kernel before sampling, the first externally-timestamped registration in either paper; 495 calls). **Control-arm floor: TRIGGERED** — no proposal at any rung scored above the seed parent (mean accepted steps 0.0 at Q4_K_M against a registered ≥ 1.0 floor), so the search-progress primaries and the conditional-quality primary are **UNINFORMATIVE** by design of the floor clause; this wave produced 109 Q2_K departures and zero improving ones, answering a harsher question about the task than about the rungs. A disclosed seed-calibration pilot (`wave8-seed-pilot` (public Kaggle kernel; owner handle withheld for double-blind review), Q4_K_M only) tested easier seeds (scores 0.000308–0.003544 against this wave’s 0.009087): the control rung improved on 1 of 40 probes even from best-of-1 random configurations, echoing 6 of 8 valid outputs at the easiest seed — the floor is the task, not the seed choice, so the recalibrated wave the pilot was scoped for was not run. **Echo (registered as a conjunction, not floor-gated): REFUTED as registered, with one side held.** Q2_K echo among valid: $186/295 = 63.1\%$, against the $\geq 55\%$ bound — held, transferring the parent-copying signature to a task whose parent is not a template. Q4_K_M: $53/113 = 46.9\%$, against the $\leq 30\%$ bound — refuted, the wave’s informative surprise: on a task where nothing climbs, the upper rung also regresses toward copying, at three times its circle-packing rate. The rung separation survives descriptively (63.1% vs 46.9%, two-sided Fisher $p = 3.5 \times 10^{-3}$, unregistered), but the clean §3 bands do not transfer intact — echo is jointly driven by precision and by whether the search has anywhere to go. The disconfirmation clause (requiring both primaries failing at Q2_K) does not fire, since the echo side held. Template check: 15.3% of Q2_K and 23.9% of Q4_K_M valid outputs score exactly zero — a fraction of the 97.5% the strongest hosted tier produced on the identical task and seed parent (§6): same task, same parent, same stall, but the quantized ladder’s dominant failure is copying while the hosted tiers’ is template emission. One tooling defect, caught by the released analysis: the kernel’s in-flight summary counted a phantom “1 accepted step” per lineage from a rounding artifact (an echo row’s rounded 12-dp score sits 5e-13 above the unrounded seed constant); exact recomputation and all 33 kernel state files agree the true count is zero everywhere.

Wave 5 (`wave5_prereg_labs.md`, LABS $n = 32$, a discrete task with no float arithmetic, registered to test the “echo is float-rounding” reading; 495 calls) returned **UNINFORMATIVE** by its own triggered control floor (Q4_K_M also accepted zero steps); no branch is claimed. Unclaimable descriptives: echo 4.3% at Q2_K vs 20.0% at Q4_K_M — **reversed**, on a task where a single flipped symbol is a complete departure. Two of three transfer tasks were too hard for the model to climb — a fact about task calibration, disclosed as such, predicted in the LABS registration’s own §8b before running.

Wave 7 (`wave7_prereg_families.md`, SHA-256 44244005..., Llama-3.1-8B-Instruct and Gemma-2-9B-it, per-family directional contrasts only, 20-valid-row power floor, disconfirmation clause requiring both families evaluable-and-refuted) returned **UNDERPOWERED both arms**: Llama produced 1 valid packing in 50 at Q4_K_M and 5 in 50 at Q2_K (same wrong-count lottery as 7B Qwen), Gemma 14 and 4. Unclaimable descriptive: **all 18 of Gemma’s valid outputs at both rungs are coordinate-exact parent echoes**, while none of Llama’s 6 are. The disconfirmation clause does not fire. One registration defect, disclosed because the file is public and immutable: its informational prior-state section quotes Qwen figures from working notes (“94%/60%/51% against 15%/10%/24%”) where ledger replay gives 94%/15% for the registered wave but **79% (19/24) against 6% (1/17)** for the fresh wave and 75% against 43% for the IQ2 pair — the discrepancy sits in a priors section referenced by no registered bound, floor or verdict. Wave 7 establishes a scope fact: the 8–9B open-weight class sits below this task’s competence floor at every precision tested.

Wave 7b (`wave7b_prereg_families_14b.md`, SHA-256 cd043d63..., Phi-4 14.7B and Mistral-Small-24B, competence-matched retry) again missed the floor: Phi-4 produced 9 valid packings in 100 loop calls, Mistral 2. **The registered both-underpowered branch holds: the task’s format-competence floor excluded every non-Qwen family tested, at up to 24B**, and family-generalty is stated as **unresolved**. Two descriptives: the competence-matched arms *climbed* (four of Phi-4’s five Q2_K lineages improved past seed, best 1.30, one Q4_K_M lineage reaching 2.1667; Mistral improved one lineage per rung, Q2_K best 2.1667, the largest climb any non-Qwen arm produced), and neither model echoed even once at either rung (Phi-4’s must-differ probe returned instruction-sensitive, vacuously, having produced no loop echo to explain). Across all four non-Qwen arms the 35 valid outputs split into Gemma’s 18 (every one a parent echo) and the other three families’ 17 (not one) — parent-copying looks family-shaped on these small samples, precisely what remains untested at adequate power.

Wave 7c (`wave7c_prereg_screened_families.md`, SHA-256 b1fc9ee9..., public Kaggle dataset pushed before sampling) changed the selection procedure, not the test: candidates were chosen by a disclosed competence screen — 50 one-shot generation-0 proposal calls per model through OpenRouter free-tier aliases at whatever precision each provider serves, wave-7b prompt constructor, parser and evaluator byte-copied and self-tested (`screen_s_doc.md`, committed before screen sampling). The screen is a selection instrument only: its serving path is unattested, so a pass gates a local pinned-weight ladder and a fail gates nothing, and no screen number is evidence. Five non-Qwen candidates screened: gemma-4-31b-it 43/50 valid, nemotron-3-super-120b-a12b 34/50, gpt-oss-20b 27/50 (all ADVANCE at the registered $\geq 20/50$ bar), nemotron-3-nano-30b-a3b 12/50 with 37 of 50 rows truncation-censored, and north-mini-code 1/50 with **49 of 50 rows returned as reasoning-censored empties** — the same silent output-budget failure mode our own Arm V amendment documents, here reproduced across a vendor boundary. The 120B advancer exceeds free-tier hardware and is recorded as advanced-not-run. The screen also logged, per row, which upstream provider served each call, and the ledger (`screen_s_raw.jsonl`) shows a single unchanged model alias served by **four different providers within one 50-call run** (DeepInfra 32, CoreWeave 14, Crusoe 2, OpenInference 2) after the model’s only free-tier provider rate-starved the registered attempt (13 content rows across the 54 calls that attempt logged; both passes kept in the ledger) — the serving heterogeneity §5 argues an alias leaves unattested, observed live and logged. On the ladder itself, gpt-oss-20b (MXFP4-native release; the Q2_K/Q4_K_M rungs are community GGUF conversions, a qualifier registered before the run) returned **UNDERPOWERED**: 5 valid packings in 50 loop calls at Q4_K_M — **every one a coordinate-exact parent echo** — and 0 in 50 at Q2_K, against the same 20-valid floor (`wave7c_analysis.py`, replay clean, state-file agreement 10/10). The screen-versus-ladder validity gap (27/50 through the alias at served precision, 5/50 on pinned Q4_K_M weights) is consistent with either conversion damage to an MXFP4-native model or served-precision sensitivity, and is not resolved here; echo-at-Q4_K_M adds a new point to the family-shaped-echo pattern above. gemma-4-31b-it, the strongest advancer, ran the same ladder (seeds

7501–7505; a first kernel attempt was cancelled at the platform’s 12-hour ceiling having spent it entirely on model download and produced zero samples — an infrastructure loss disclosed for completeness. The retry required a post-registration runner edit — confined to the download path, and verifiable: the registered runner survives at commit `8cfa9fb` and the retry at `342cdea`, whose runner diff is 18 lines — a one-line `aria2` install in setup plus 17 lines replacing the fetch helper inside the model-fetch function; seeds, prompts, sampling parameters and every line of sampling code are unchanged between the two. No amendment was registered before the retry — unlike this corpus’s arm-level serving-path deviations, which each received one pre-resumption — so the commit-pair diff stands as an after-the-fact substitute, and the gap in amendment discipline is named here rather than left for a reader to find). It returned the wave’s only informative verdicts. 7c.1, the primary echo contrast: **UNDERPOWERED** — Q2_K produced 0 valid packings in 50 against the 20-valid floor (Q4_K_M: 24), no branch claimed. 7c.2, the floor-gated search-progress secondary: the Q4_K_M control floor *passed* — the first non-Qwen arm in any of this programme’s waves to clear one (mean accepted steps **4.800** against the ≥ 1.0 floor, all five seeds improving, best 2.08 on every seed) — and the prediction **HELD: 0.000** accepted steps at Q2_K against 4.800 at Q4_K_M. 7c.3, the must-differ probe at Q2_K: **UNDERPOWERED** (0 of 10 probes valid; the Q4_K_M side returned 10 of 10 valid with 0 echoes-under-differ). The descriptive shape differs from every family before it: at Q2_K gemma-4 stays *viable* — 48 of 50 loop calls returned parseable proposals, identical to its Q4_K_M viability — while producing nothing valid and echoing nothing. Where Qwen’s 2-bit collapse is parent-copying and gemma-2’s 18 valid outputs were all echoes, gemma-4’s is well-formed invalidity: a third failure channel, equally invisible to viability-style health metrics — a channel taxonomy that is descriptive and unregistered, like every cross-family comparison in this section. Echo at Q4_K_M: 0/24. Gemma-4’s rungs are community GGUF conversions like every dense wave’s (from a BF16-native release, so gpt-oss’s MXFP4 qualifier does not apply); whether the Q2_K invalidity reflects 2-bit precision or conversion quality for this architecture is not disentangled here — the attribution qualifier every K-quant ladder in this paper shares. The disconfirmation clause (7c.1 refuted in both families, both evaluable) does not fire — neither family’s 7c.1 was evaluable. What wave 7c adds to family generality is therefore one directional result: a second model family, the first competent enough at 4-bit for its control floor to pass, accepts nothing at 2-bit — reproducing the loop-level search collapse of §3.6 under a registered, floor-gated prediction. It is one held secondary among the many predictions this programme has registered across its six generality waves, most of which returned **UNDERPOWERED** or **UNINFORMATIVE**, and it is reported as one directional replication, not a resolved generalization — the echo-contrast primary that carries §3’s headline remains unresolved beyond Qwen at every power tested, and the *channel* of 2-bit failure now looks family-shaped on small samples: echo in Qwen and gemma-2, invalidity in gemma-4, echo again in gpt-oss’s five.

Finally, §3’s data comes from a locally-served open-weights ladder at $N = 26$, not from the alias-addressed runtime and not at §4’s cells ($N = 13, 21, 31$). The mechanism gap — that served quantization matters for this task — is supported by §3’s measurement rather than measured inside the runtime, which is exactly the measurement §5 says cannot be made. The instance gap — that a cliff observed at $N = 26$ transfers to $N = 13/21/31$ — is supported by the companion paper’s finding that the same constructible template family governs proposals across N , but is an inference, not a replication. A reader should treat §3 as establishing that the failure mode *exists and is invisible to pass/fail metrics*, and treat its quantitative rungs as belonging to the sweep, not to §4.

Use of AI systems

This paper studies language-model behaviour and was written with language-model assistance. Claude models wrote the collection, scoring and analysis scripts and drafted the manuscript prose; the human author directed the programme, approved each preregistration before sampling, made the final inclusion, stopping and submission decisions, and is solely responsible for the content. Internal adversarial checks by language models of other families preceded submission; they are quality assurance, not review, and carry no external authority. The claim a reader can check is the ordering, not the authorship — each preregistration commit is a git ancestor of the sampling it governs — and because the authoring models share a family with the runtime under study in §4, the released scripts, raw ledgers and frozen outputs, rather than the authorship, are what the claims rest on.

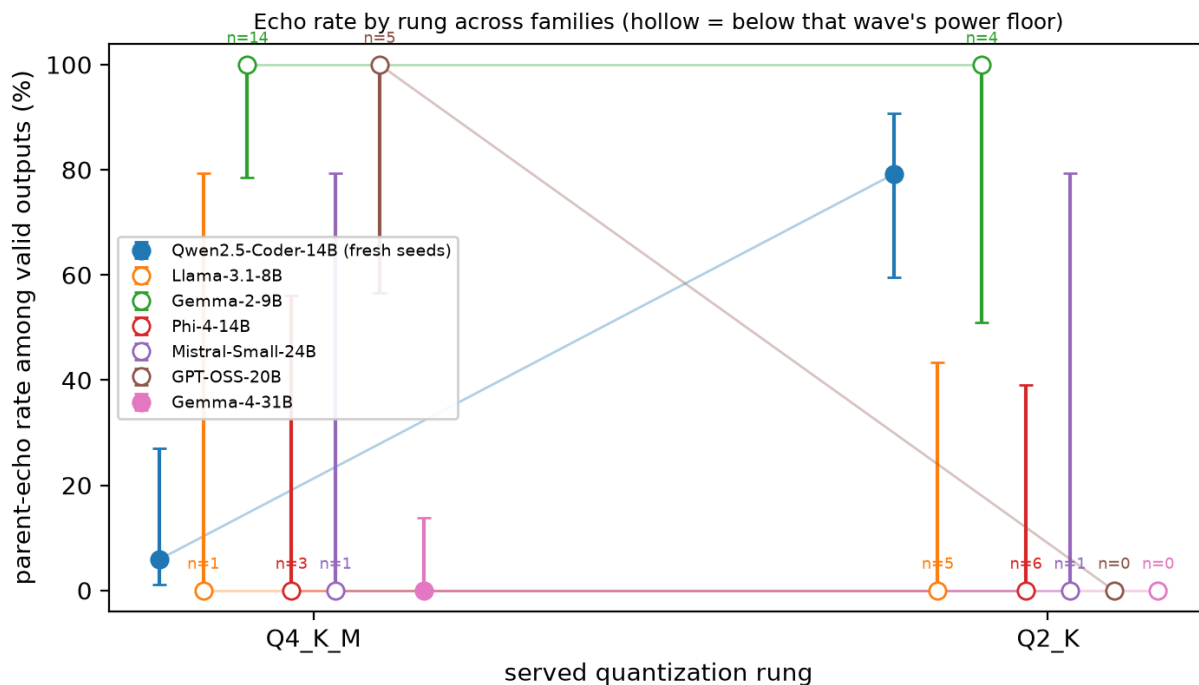


Figure 1: Coordinate-verified parent-echo among valid outputs, by rung, across every family ladder in the programme, replayed from the raw ledgers (each family judged against its own registration’s power floor). Qwen’s fresh-seed wave carries the registered contrast (1/17 at Q4_K_M against 19/24 at Q2_K); gemma-2 echoes every output it validates at both rungs (14/14, 4/4); gpt-oss echoes all five of its Q4_K_M validities; gemma-4 echoes none of its 24 and validates nothing at Q2_K; Llama, Phi-4 and Mistral sit below their competence floors. Across the families that clear their floors the pattern is consistent with family-shaped echo and family-shaped 2-bit failure channels; the below-floor bars cannot speak to either, and no bar here is a registered verdict.

Claim–evidence map

Every row names a file a reader can open **inside the released repository**. Earlier drafts satisfied the first half of that sentence and not the second: every §3 row pointed into `../agent-run/` or `../handoff/`, sibling trees on the authoring host that no clone contains, so a reader could see the path and not the data. The §3 artifacts — both dispersion-probe waves with their preregistrations, provenance and analyser, and the four ladder output trees — are now vendored under `sec3_artifacts/` (95 files, 1.8 MB), and the released scripts default to those in-repo copies, so `python sec3_dispersion_registered.py` and `python sec3_artifacts/dispersion_probe_v2/analyze_v2.py` both run in a fresh clone with no arguments. Dependencies are declared in `requirements.txt`. Where a figure is *not* checkable from the released artifacts, the row says so instead of pointing at a document that merely repeats it.

Table 2: Claim–evidence map: each claim paired with a file a reader can open inside the released repository.

Claim	Evidence
§3 14B ladder: viability 22/22/24/19, validity 18/20/19/18, echo 2/18, 3/20, 3/19 and 17/18, per-seed vectors, invalid-row near-copy fractions	<code>sec3_artifacts/precision_sweep_14b_v2_output/**/candidates_precision_14b.jsonl</code> (200 rows), replayed by <code>sec3_ladder_repro.py</code>

continued on next page

Claim	Evidence
§3 fresh-seed replication: 19/24 vs 1/17, per-seed 4/5, 4/5, 5/6, 2/2, 4/6	<code>sec3_artifacts/precision_sweep_14b_fresh_output/**/candidates_precision_14b_fresh.jsonl</code> (100 rows), same script
§3 must-differ probe: 5/5 at Q2_K, 1/5 at Q4_K_M; 6/8 vs 0/2 on the IQ2 pair	<code>sec3_artifacts/**/mustdiffer_14b.jsonl</code> , <code>mustdiffer_14b_iq2.jsonl</code> ; echo computed against the fixed seed parent, no echo flag in the ledger
§3 IQ2 control: 24/32 vs 12/28	<code>sec3_artifacts/precision_sweep_14b_iq2_output/**/candidates_precision_14b_iq2.jsonl</code> (100 rows), same script
§3 fixed-parent dispersion control: 34/65, 31/51, 46/49 and the parent-score gradient	<code>sec3_artifacts/dispersion_probe/probe_samples.jsonl</code> (288 rows), echo field as labelled
§3 served weight files (SHA-256, byte length, repo, sampling parameters, GPUs)	<code>provenance.json</code> in each of the three ladder output directories
§3 seeded parent (26-circle 6 × 5 grid, score 0.89999)	<code>sec3_artifacts/precision_sweep_14b_v2_output/**/state/q2_k_seed_42.json</code> , a lineage that never improved
§3 Fisher figures recomputed by a released script : $p = 1.7 \times 10^{-7}$, 3.4×10^{-6} , 0.007, 0.017, 0.056	<code>sec3_ladder_repro.py</code> prints each as a Fisher check line beside the paper’s rounded value
§3 Fisher figures $p = 5.7 \times 10^{-10}$ (17/18 vs 8/57), $p = 0.001$ (15/15 vs 1/5, pooled), $p = 0.44$ (F1’s outcome), $p = 1.0$ (FP16 vs Q3_K_M viability)	<code>sec3_7b_repro.py</code> , run with no arguments. It also prints the paired form of the 0.001 contrast, $p = 0.048$, so the pooling is visible
§3 inferential status of all twenty-nine p -values (accounting in Appendix A.4)	§3: nine Fisher, six preregistered permutation tests, two Spearman orthogonality diagnostics and twelve post-hoc lineage-level permutation tests; the four families not pooled, the reason given for each, and the Bonferroni threshold stated for the Fisher and post-hoc families
§3.6 search progress: accepted steps per lineage (1/50, 15/50, 16/50, 14/50 on the registered ladder; 3/50 vs 14/50 fresh; 6/50 vs 16/50 IQ2; 7/50, 4/50, 3/50, 2/50, 6/50 at 7B), the identity gaps k , the conditional-on-departure rates, final best per lineage, and all twelve exact permutation tails	<code>sec3_search_progress.py</code> , run with no arguments; replays all four vendored ledgers. The two 7B final-best tails, 0.158 and 0.119, are stated in §3.6’s 7B paragraph. Post-hoc — labelled as such at every use, and §3.6 states that the fresh wave’s registered <i>primary</i> (F1) failed
§3.6 conditional quality: departure counts (10 in 133 valid at Q2_K against 74 in 164 at Q4_K_M) and the per-row <code>score_delta</code> data behind the open departure-quality question	<code>sec3_conditional_quality.py</code> , run with no arguments, from <code>score_delta</code> and <code>echo</code> in the two vendored probe ledgers. Post-hoc ; UNDERPOWERED against a 25-departure floor from an unlocked design note, so §3.6 reports the departure-quality question as open
§3.6 horizon argument: departure rates and offer distributions (6 vs 33 observations), the discarded directional projection, and the withdrawn power figure	<code>sec3_horizon_power.py</code> , run with no arguments. A resampling projection , not a measurement; detail in Appendix C
§3.6 the failed registered improvement prediction F1 ($\leq 2/5$ Q2_K, $\geq 4/5$ Q4_K_M; observed 3/5 vs 5/5) and its primary designation	<code>sec3_artifacts/runners/kaggle_precision_sweep_14b_fresh.py</code> header, which reads F1 Improvement (seed-level, primary); F2, the echo bound the paper leans on, carries no such label. Outcome computed by <code>sec3_search_progress.py</code> from the fresh ledger
§6 wave 4 capability-tier control: echo 0/120 (0% per tier vs registered $\leq 30\%$ bound, verdict DISSOCIATION), secondary 5.2 NOT EVALUATED (10-echo floor unmet), template rates 52.5% / 27.5% / 97.5%, zero improvements past seed, <code>tool_uses</code> 0/120	<code>wave4_artifacts/wave4_tiers.jsonl</code> (120 rows), replayed by <code>wave4_analysis.py</code> with no arguments, which recomputes every echo and score from stored coordinates and prints each verdict label. Registration <code>wave4_prereg_tiers_heilbronn.md</code> (SHA-256 7b3e67dc..., public Kaggle dataset <code>wave4-prereg-tiers-heilbronn</code> (owner handle withheld for double-blind review), pushed before sampling). All-distinct uniqueness count is post-hoc, from wave4_artifacts/MODEL_IDS_NOTE.md , not computed by the replay script; model-ID caveats same file
§3 7B ladder: viability 7/6/9/7/16 per 50 with Wilson intervals, all three contrasts, the format-lottery counts (45 of 250 at exactly 26, 153 at 24-25, modal count per rung), the truncation check (687 tokens against a 1200 cap among the 24-25 rows), probes 0/30, best score 1.79998	<code>sec3_7b_repro.py</code> , run with no arguments, from <code>sec3_artifacts/precision_sweep/</code> . Narrative and scope caveats in <code>sec3_artifacts/precision-cliff-paper-combined.md</code> §3.4, §5.9, §6
§3 IQ2_M invalid near-copy 2/22	not reproduced : our replay of the stated definition returns 1/22 (§3)

continued on next page

Claim	Evidence
§8 wave 3 (Heilbronn): control floor TRIGGERED → 5.1/5.2/5.3 UNINFORMATIVE; 5.4 echo Q2_K 186/295 = 63.1% HELD vs $\geq 55\%$, Q4_K_M 53/113 = 46.9% REFUTED vs $\leq 30\%$, conjunction REFUTED; Fisher 3.5×10^{-3} (descriptive); template 15.3%/23.9%; 0/495 improvements; phantom-step rounding artifact	wave3_output/precision_sweep_14b_heilbronn/candidates_precision_14b_heilbronn.jsonl (495 rows), replayed by wave3_analysis.py with no arguments in exact arithmetic; agreement with all 33 kernel state files printed. Registration wave3_prereg_heilbronn.md + runner SHA 101b298c..., pushed as public Kaggle kernel precision-sweep-14b-heilbronn-wave3 (public Kaggle kernel; owner handle withheld for double-blind review) before sampling
§8 wave 5 (LABS): control floor TRIGGERED → verdict UNINFORMATIVE, Branch C, no branch claimed; unclaimable descriptives echo 10/233 = 4.3% Q2_K vs 12/60 = 20.0% Q4_K_M (reversed), 0 improving departures anywhere, 0 symmetry variants	wave5_output/precision_sweep_14b_labs/ ledger (495 rows), replayed by wave5_analysis.py with no arguments in exact integer arithmetic. Registration wave5_prereg_labs.md SHA a1336cfd..., runner SHA d1739f6c..., pushed as public kernel precision-sweep-14b-labs-wave5 (public Kaggle kernel; owner handle withheld for double-blind review) before sampling
§8 wave 7 (family generality): UNDERPOWERED both families; control floors fired; disconfirmation clause does not fire; unclaimable descriptives — gemma 18/18 valid outputs are parent echoes at both rungs, llama 0/6	wave7_output/wave7_{llama31_8b,gemma2_9b}/ ledgers (100 rows each), replayed by wave7_analysis.py with no arguments. Registration wave7_prereg_families.md SHA 44244005..., public Kaggle dataset wave7-prereg-families (owner handle withheld for double-blind review) pushed before sampling
§8 wave 7b (competence-matched retry): UNDERPOWERED both families → registered both-underpowered branch holds; unclaimable descriptives — phi-4 0 echoes, 4/5 Q2_K lineages improved (best 1.30), must-differ instruction-sensitive (vacuously); mistral 0 echoes, 1 lineage improved per rung, Q2_K best 2.1667	wave7b_output/wave7b_{phi4_14b,mistral24b}/ ledgers (100 rows each), replayed by wave7b_analysis.py with no arguments. Registration wave7b_prereg_families_14b.md SHA cd043d63..., public Kaggle dataset wave7b-prereg-families-14b (owner handle withheld for double-blind review) pushed before sampling
§8 wave 7c (screen-selected retry): screen tally gemma-4-31b 43/50, nemotron-super 34/50 (advanced-not-run, hardware), gpt-oss 27/50 ADVANCE; gpt-oss ladder UNDERPOWERED (5/50 valid Q4_K_M — all parent echoes — 0/50 Q2_K, MXFP4-conversion qualifier registered); north-mini 49/50 reasoning-censored (non-evidential); provider churn: 4 providers, one alias, one run; gemma-4-31b ladder: 7c.1 UNDERPOWERED (Q2_K 0 valid vs floor 20, Q4_K_M 24), 7c.2 HELD (accepted steps 0.000 Q2_K vs 4.800 Q4_K_M, control floor passed, all 5 seeds improved, best 2.08 every seed), 7c.3 UNDERPOWERED (0/10 valid probes Q2_K; Q4_K_M 10/10 valid, 0 echo-under-differ); descriptives: viability 48/50 both rungs, Q4_K_M echo 0/24; attempt-1 kernel cancelled at 12-h ceiling with zero samples (disclosed)	screen_s_raw.jsonl (304 rows, every attempt: 50 per screened model plus 54 for the rate-starved gemma free-tier pass; the registration's own §SS2 cites 322 — a miscount, not a mutation: the ledger's first commit 4f618c7 already held exactly these 304 rows — a registration defect in a descriptive count referenced by no bound or verdict, disclosed because the file is public and immutable), wave7c_output/wave7c_{gpt_oss_20b,gemma4_31b}/ ledgers replayed by wave7c_analysis.py with no arguments. Registration wave7c_prereg_screened_families.md SHA b1fc9ee9..., public Kaggle dataset wave7c-prereg-screened-families (owner handle withheld for double-blind review) pushed before sampling; screen instrument screen_s_doc.md committed f037c34 before screen sampling
§3 re-execution determinism, externally exercised: 112/112 per-row digest MATCH on T4 x2 (verdict SUPPORTED); two prior P100 draws returned 112/112 VOID_ARCH each, not compared	kaggle_redetermin/ runner (SHA 58d9a4c8..., 112 embedded per-row digests from the vendored v2 ledger) + output_v3/redetermin_summary.json; public kernel precision-redetermin (public Kaggle kernel; owner handle withheld for double-blind review). Caveats in-file: build_unverified: true (wheel version-pinned 0.3.34); q8_0/q3_k_m skipped with reasons printed
§6 item 4 self-audit and the failed repair: duration CV 0.0603 vs 0.0143 (wave 1) and 0.0692 vs 0.0200 (wave 2), ratios below the item's own one-third threshold, permutation $p < 0.0001$; the echo/non-echo CV split; wave-2 throughput ranges disjoint and wave-1 ranges overlapping ; the $\sim 92\%$ cold-start ratio; the same-SHA hardware inversion (Q2_K 16.62 tok/s on P100 vs 22.48 on $2 \times T4$); the length dependence; the bit-width non-monotonicity §4 cites	sec6_cv_canary_audit.py, run with no arguments. It reads both probe ledgers and both 14B/7B ladder ledgers . The serving path is fixed by construction across the probe rows, so the firing is a false positive by design of the comparison
§4 validity, failure taxonomy, families, scores, both tolerances	arm_f_raw.json opus_alias / sonnet_bare / bare rows, recomputed with arm_f_repro.py
§4 fourth comparator: trace 63/70 valid (18/20, 28/30, 17/20), Fisher 1.1×10^{-13} against opus_alias and 1.0×10^{-16} pooled, comparator-vs-comparator $p = 0.30$ and 0.099	arm_f_raw.json rows with arm == "trace", scored by sec4_independent_rescore.py's checker. Pools the companion paper's pilot and trace_v2 rows, which that paper forbids pooling — stated in §4, no claim rests on the arm
§4 durations	STATE.md §8, §8b — session-log ranges; per-invocation vector not captured (§4, §5)

continued on next page

Claim	Evidence
§4 registration digest	<code>arm_o_preregistration.txt</code> , SHA-256 published in §4
§4.1 arm B2 re-snapshot: 10/10, 10/10, 10/10 against 3/10, 1/10, 0/10; pooled 30/30 vs 4/30, Fisher $p = 7.8 \times 10^{-13}$; best valid 1.776141, 2.276800, 2.794180; latency 68.7-594.1 s; usage 58k-195k	<code>arm_r_artifacts/b2_alias_n{13,21,31}.jsonl</code> , replayed by <code>arm_r_analysis.py</code> with no arguments
§4.1 arm R repair probe: 59/60 exact, per-cell 5/5 everywhere except alias <code>n31_d0.001</code> at 4/5, recall 1.00 in every cell, zero empty responses, and the single miss's 7.089e-04 true clearance	<code>arm_r_artifacts/r_n{13,31}.jsonl</code> against the exact key in <code>arm_r_key.json</code> , both replayed by <code>arm_r_analysis.py</code>
§4.1 registered scorecard: P-R1, P-R4, P-R5 held (P-R4 vacuously); P-R0, P-R2, P-R3, P-R6 failed	<code>arm_r_preregistration.md</code> , committed in e77bb0e before any sampling; scored line by line in <code>arm_r_analysis.py</code> . A local commit, not an external timestamp
§5 prompt digests, $N = 13$ and $N = 31$	<code>arm_f_repro.py</code> <code>prompt_hash()</code> , <code>arm_f_prompts.json</code>
§5 prompt digest, $N = 21$	absent from <code>arm_f_prompts.json</code> but derivable: substituting 21 into the released $N = 13$ prompt reproduces the digest exactly (§5)
§3 probe's registered centres-echo, the registered primary median-displacement measure, and the v2 wave's counts	<code>sec3_artifacts/dispersion_probe/</code> , <code>sec3_artifacts/dispersion_probe_v2/</code> , recomputed by <code>sec3_dispersion_registered.py</code> ; the median measure's registered JT test is in <code>sec3_registered_echo_test.py</code>
§3 probe's registered JT/NED test ($p = 0.030$), rarefaction count ($p = 0.953$), quality fork ($p = 0.454$), and the rule's unclassified verdict	<code>analysis_prereg.md</code> ; computed from <code>raw_text</code> and <code>circles</code> in the released rows. NED is a substituted definition, not the kernel's — Appendix B
§3 wave-2 registered verdict FAILED , its primary rarefaction (38.31 / 46.78 / 13.00 at $m = 84$, JT $p = 0.0081$) and score fork ($p = 0.3526$)	<code>sec3_artifacts/dispersion_probe_v2/analysis_prereg_v2.md</code> + <code>analysis_prereg_v2_addendum.md</code> ; printed by <code>sec3_artifacts/dispersion_probe_v2/analyze_v2.py</code> , run with no arguments
§3 wave-1 registered primary echo JT ($p = 0.686$, 159/165 rows at exactly zero), registered <code>score_delta</code> CIs, and the orthogonality p -values	<code>sec3_registered_echo_test.py</code> , run with no arguments; §2 of <code>sec3_artifacts/dispersion_probe/analysis_prereg.md</code> Amendment 1 designates this measure the primary
§3 ladder registrations (fresh-seed bound <code>q2_k >= 60%</code> , <code>q4_k_m <= 35%</code> ; must-differ decision rule; improvement-count prediction)	<code>sec3_artifacts/runners/kaggle_precision_sweep_14b_fresh.py</code> and <code>..iq2.py</code> , in the header comment block pushed before the run executed
§3 post-hoc decomposition, both waves (46/31/4% and 39/51/6%)	emitted by <code>sec3_dispersion_registered.py</code> ; labelled post-hoc at every use
§3 probe's design limit (six parents, one shared lattice) and Q8_0's skipped condition	<code>sec3_artifacts/dispersion_probe/probe_samples.jsonl</code> (parent centre lists identical across all six <code>parent_id</code> values) and both waves' <code>provenance.json</code> , which record "q8_0": {"skipped": "needs 2 gpus, have 1"}
§3 probe's 18 harness-error rows	<code>sec3_artifacts/dispersion_probe/probe_samples.jsonl</code> , rows whose <code>parse_error</code> reads <code>gen_error: ValueError: logprobs is not supported</code> ; six per rung. The v2 ledger has no such rows
§5 alias-map provenance and its gap	<code>ALIAS_MAP</code> , <code>RUN_DATE</code> , <code>PROPOSER_ALIAS</code> in <code>arm_f_repro.py</code>
§4 latency ranges (2.8-9 s vs 75-250 s and 150-1170 s)	not checkable : <code>STATE.md</code> §8, §8b working-log ranges only; <code>arm_f_raw.json</code> rows carry no duration field (§4, §6 item 4)
§6 item 3 worked example (32/45 → 32/50)	not checkable : the 45-row slice was overwritten in place and cannot be reconstructed (§4, §6)
§7 systems citations	<code>p8_systems_citations.md</code> (self-certified)
§7 hazard-literature citations (2307.09009, 2603.19022, 2607.10252, 2607.08734, 2604.19884, 2509.19349, GUIDE-LLM, 2508.15503, 2601.01954, 2512.00651)	each checked against its live arXiv or publisher page

Broader Impact Statement

This paper reports a way for a quantized proposer to keep every health metric a discovery loop watches while the search underneath it collapses. Two readings would be misuses of that result. It is not an argument against quantization: quantized serving is what puts capable models within reach of researchers without datacentre budgets, and nothing here shows harm at the rungs most practitioners use — the collapse we measure is at the 2-bit endpoint, on one task family, at one scale. Nor is it an argument that published loop results are wrong; they are unrepeatable in a specific and now-demonstrable way, which calls for disclosure rather than retraction. The costs of the repair protocol in §6 fall hardest on exactly the researchers least able to absorb them, and we have tried to keep every item in it cheap and local.

Two further repercussions are worth naming. The forensic method of §4 — reading a serving path from latency and behaviour — is a capability as well as a diagnostic, and the same reasoning that lets a researcher

ask what served their calls could let a third party probe an endpoint its operator would rather not have characterized; we think the transparency gain dominates here, because the alternative is a research literature that cannot audit its own substrate, but the direction is not costless. Finally, this work addresses a commercial alias whose behaviour changed mid-study. We report that as an observability failure of the addressing mode, common to managed runtimes generally, and not as a claim about any vendor’s conduct; we have no evidence of intent and make no allegation of any.

References

- Sebastian Baltes, Florian Angermeir, Chetan Arora, Marvin Muñoz Barón, Chunyang Chen, Lukas Böhme, Fabio Calefato, Neil Ernst, Davide Falesi, Brian Fitzgerald, Davide Fucci, Junda He, Christoph Treude, Marcos Kalinowski, Stefano Lambiase, Daniel Russo, Mircea Lungu, Cristina Martinez Montes, Lutz Prechelt, Paul Ralph, Rijnard van Tonder, and Stefan Wagner. Guidelines for empirical studies in software engineering involving large language models, 2026. URL <https://arxiv.org/abs/2508.15503>.
- Tomas Bruckner. One token is enough: Fingerprinting and verifying large language models from single-token output distributions, 2026. URL <https://arxiv.org/abs/2607.10252>.
- Will Cai, Tianneng Shi, Xuandong Zhao, and Dawn Song. Are you getting what you pay for? auditing model substitution in llm apis, 2025. URL <https://arxiv.org/abs/2504.04715>.
- Mert Cemri, Shubham Agrawal, Akshat Gupta, Shu Liu, Audrey Cheng, Qiuyang Mang, Ashwin Naren, Lutfi Eren Erdogan, Koushik Sen, Matei Zaharia, Alex Dimakis, and Ion Stoica. Adaevolve: Adaptive llm driven zeroth-order optimization, 2026. URL <https://arxiv.org/abs/2602.20133>.
- Ting-Yun Chang, Muru Zhang, Jesse Thomason, and Robin Jia. Why do some inputs break low-bit llm quantization?, 2025. URL <https://arxiv.org/abs/2506.12044>.
- Lingjiao Chen, Matei Zaharia, and James Zou. How is chatgpt’s behavior changing over time?, 2023. URL <https://arxiv.org/abs/2307.09009>.
- Stefan Feuerriegel, Kerstin Forster, Dominique Geissler, Abdurahman Maarouf, Sebastian Maier, Barbara Plank, et al. A reporting checklist for large language models in behavioural science. *Nature Human Behaviour*, 10:1182–1186, 2026. doi: 10.1038/s41562-026-02492-7. GUIDE-LLM.
- Elias Frantar, Saleh Ashkboos, Torsten Hoeffler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers, 2023. URL <https://arxiv.org/abs/2210.17323>.
- Dan Friedman and Adji Bousso Dieng. The vendi score: A diversity evaluation metric for machine learning, 2023. URL <https://arxiv.org/abs/2210.02410>.
- Horace He et al. Defeating nondeterminism in LLM inference. Thinking Machines Lab blog, September 2025. URL <https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/>.
- Haozhe Jia. Hindsightbench: A black-box behavioral audit protocol for parametric hindsight in time-indexed llm decision tasks, 2026. URL <https://arxiv.org/abs/2607.18867>.
- Valentin Khurlov, Andrey Galichin, Denis Bashkirov, Dmitry Vinichenko, Oleg Travkin, Roman Alferov, Andrey Kuznetsov, and Ivan Oseledets. Gigaevolve: An open source optimization framework powered by llms and evolution algorithms, 2025. URL <https://arxiv.org/abs/2511.17592>.
- Alexander Korn, Lea Zaruchas, Chetan Arora, Andreas Metzger, Sven Smolka, Fanyu Wang, and Andreas Vogelsang. Reporting llm prompting in automated software engineering: A guideline based on current practices and expectations, 2026. URL <https://arxiv.org/abs/2601.01954>.
- Uygar Kurt. Which quantization should i use? a unified evaluation of llama.cpp quantization on llama-3.1-8b-instruct, 2026. URL <https://arxiv.org/abs/2601.14277>.
- Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. Shinkaevolve: Towards open-ended and sample-efficient program evolution, 2025. URL <https://arxiv.org/abs/2509.19349>.

- Joel Lehman and Kenneth O. Stanley. Abandoning objectives: Evolution through the search for novelty alone. *Evolutionary Computation*, 19(2):189–223, 2011.
- Jonah Leshin, Manish Shah, Ian Timmis, and Daniel Kang. Behavioral fingerprints for llm endpoint stability and identity, 2026. URL <https://arxiv.org/abs/2603.19022>.
- Jiwei Li, Michel Galley, Chris Brockett, Jianfeng Gao, and Bill Dolan. A diversity-promoting objective function for neural conversation models, 2016. URL <https://arxiv.org/abs/1510.03055>.
- Zhen Li, Yupeng Su, Runming Yang, Congkai Xie, Zheng Wang, Zhongwei Xie, Ngai Wong, and Hongxia Yang. Quantization meets reasoning: Exploring llm low-bit quantization degradation for mathematical reasoning, 2025. URL <https://arxiv.org/abs/2501.03035>.
- Zhen Li, Yupeng Su, Songmiao Wang, Runming Yang, Congkai Xie, Aofan Liu, Ming Li, Jiannong Cao, Yuan Xie, Ngai Wong, and Hongxia Yang. Quantization meets reasoning: Exploring and mitigating degradation of low-bit llms in mathematical reasoning, 2026. URL <https://arxiv.org/abs/2505.11574>.
- Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for llm compression and acceleration, 2024. URL <https://arxiv.org/abs/2306.00978>. MLSys 2024.
- Jianzhe Lin. Self-improvement can self-regress: The rise-and-collapse failure mode of llm self-training, 2026. URL <https://arxiv.org/abs/2606.21090>.
- Haiquan Lu, Zigeng Chen, Gongfan Fang, Xinyin Ma, and Xinchao Wang. Mix-quant: Quantized prefilling, precise decoding for agentic llms, 2026. URL <https://arxiv.org/abs/2605.20315>.
- Sichun Luo, Yi Huang, Haochen Luo, Fengyuan Liu, Guanzhi Deng, Lei Li, Qinghua Yao, Zefa Hu, Junlan Feng, and Qi Liu. Seaevo: Advancing algorithm discovery with strategy space evolution, 2026. URL <https://arxiv.org/abs/2604.24372>.
- Keyu Lv, Manyi Zhang, Xiaobo Xia, Jingchen Ni, Shannan Yan, Xianzhi Yu, Lu Hou, Chun Yuan, and Haoli Bai. What makes low-bit quantization-aware training work for reasoning llms? a systematic study, 2026. URL <https://arxiv.org/abs/2601.14888>.
- Lovish Madaan, Aaditya K. Singh, Rylan Schaeffer, Andrew Poulton, Sanmi Koyejo, Pontus Stenertorp, Sharan Narang, and Dieuwke Hupkes. Quantifying variance in evaluation benchmarks, 2024. URL <https://arxiv.org/abs/2406.10229>.
- Kelly Marchisio, Saurabh Dash, Hongyu Chen, Dennis Aumiller, Ahmet Üstün, Sara Hooker, and Sebastian Ruder. How does quantization affect multilingual llms?, 2024. URL <https://arxiv.org/abs/2407.03211>.
- Evan Miller. Adding error bars to evals: A statistical approach to language model evaluations, 2024. URL <https://arxiv.org/abs/2411.00640>.
- Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites, 2015. URL <https://arxiv.org/abs/1504.04909>.
- Alexander Novikov, Ngan Vũ, Marvin Eisenberger, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery, 2025. URL <https://arxiv.org/abs/2506.13131>.
- Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d’Alché Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program). *Journal of Machine Learning Research*, 22(164): 1–20, 2021. arXiv:2003.12206.
- Baha Rababah, Cuneyt Gurcan Akcora, and Carson K. Leung. The illusion of equivalency: Statistical characterization of quantization effects in llms, 2026. URL <https://arxiv.org/abs/2607.08734>.

- Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.
- Christoph Schnabl, Daniel Hugenroth, Bill Marino, and Alastair R. Beresford. Attestable audits: Verifiable ai safety benchmarks using trusted execution environments, 2025. URL <https://arxiv.org/abs/2506.23706>.
- Mohammed Latif Siddiq, Arvin Islam-Gomes, Natalie Sekerak, and Joanna C. S. Santos. Large language models for software engineering: A reproducibility crisis, 2025. URL <https://arxiv.org/abs/2512.00651>.
- Chang Su, Zhongkai Hao, Zhizhou Zhang, Zeyu Xia, Youjia Wu, Hang Su, and Jun Zhu. Helix: Evolutionary reinforcement learning for open-ended scientific problem solving, 2026. URL <https://arxiv.org/abs/2603.07642>.
- Maria Thomas, Kristina Gligoric, and Nihar B. Shah. Mitigating llm-based p-hacking by preregistering for the next llm, 2026. URL <https://arxiv.org/abs/2606.27687>.
- Michelle Vaccaro. Preregistration for experiments with ai agents, 2026. URL <https://arxiv.org/abs/2606.11217>.
- Yiping Wang, Shao-Rong Su, Zhiyuan Zeng, Eva Xu, Liliang Ren, Xinyu Yang, Zeyi Huang, Xuehai He, Luyao Ma, Baolin Peng, Hao Cheng, Pengcheng He, Weizhu Chen, Shuohang Wang, Simon Shaolei Du, and Yelong Shen. Thetaevolve: Test-time learning on open problems, 2025. URL <https://arxiv.org/abs/2511.23473>.
- Marcus Williams, Hannah Sheahan, Cameron Raymond, Tomek Korbak, Deng Pan, Peilin Yang, Leon Maksin, Ningyi Xie, Phillip Guo, Ian Kivlichan, and Micah Carroll. Predicting llm safety before release by simulating deployment, 2026. URL <https://arxiv.org/abs/2607.07184>.
- Anna Wimbauer, Jonas Möller, Erik Imgrund, and Konrad Rieck. Fingerprinting inference systems of large language models, 2026. URL <https://arxiv.org/abs/2605.29979>.
- Sixue Xing, Haoyu He, Kerui Wu, Zhuo Yang, Haozheng Luo, Tianfan Fu, and Aarth Nagarajan. Compute allocation in evolutionary search: From depth-breadth to multi-armed bandits, 2026. URL <https://arxiv.org/abs/2605.29268>.
- Minghao Yan, Bo Peng, Benjamin Coleman, Ziqi Chen, Zhouhang Xie, Shuo Chen, Zhankui He, Naveen Sachdeva, Isabella Ye, Weili Wang, Chi Wang, Ed H. Chi, Fernando Pereira, Wang-Cheng Kang, Derek Zhiyuan Cheng, and Beidou Wang. Pacevolve: Enabling long-horizon progress-aware consistent evolution, 2026. URL <https://arxiv.org/abs/2601.10657>.
- Jiayi Yuan, Hao Li, Xinheng Ding, Wenya Xie, Yu-Jhe Li, Wentian Zhao, Kun Wan, Jing Shi, Xia Hu, and Zirui Liu. Understanding and mitigating numerical sources of nondeterminism in llm inference, 2025. URL <https://arxiv.org/abs/2506.09501>.
- Yuewei Zhang, Zhi-Hai Zhang, and Hanzhang Qin. Which model is actually serving you? iris: Budgeted black-box auditing of model substitution and routing dilution in llm gateways, 2026. URL <https://arxiv.org/abs/2607.20860>.
- Chenxi Zhou, Pengfei Cao, Jiang Li, Bohan Yu, Jinyu Ye, Jun Zhao, and Kang Liu. From signal degradation to computation collapse: Uncovering the two failure modes of llm quantization, 2026. URL <https://arxiv.org/abs/2604.19884>.
- Lexin Zhou, Wout Schellaert, Fernando Martínez-Plumed, Yael Moros-Daval, Cèsar Ferri, and José Hernández-Orallo. Larger and more instructable language models become less reliable. *Nature*, 634(8032):61–68, 2024. doi: 10.1038/s41586-024-07930-y.
- Yaoming Zhu, Sidi Lu, Lei Zheng, Jiaxian Guo, Weinan Zhang, Jun Wang, and Yong Yu. Texygen: A benchmarking platform for text generation models, 2018. URL <https://arxiv.org/abs/1802.01886>.

A the dispersion probes’ registered analyses, in full

Section 3 reports every registered outcome of both probe waves, with its status label attached, in the tables where it belongs. This appendix holds the derivation narrative those tables point at: what was omitted from earlier drafts, what was reconstructed and why, and which conventions are ours rather than registered.

A.1 What earlier drafts omitted from wave 2, and in which direction

Wave 2 carries `analysis_prereg_v2.md` and a truncated-run addendum. Addendum rule R2 requires a ceiling rung to land; `q8_0` never ran, so the wave’s own verdict is **FAILED** and every descriptive it produces is exploratory. Earlier drafts described the wave as “a registered confirmatory replication,” quoted two of its outputs as replicating, and reported neither the label nor the wave’s registered primary test. Both omissions cut in opposite directions: the registered **FAILED** label was omitted **against us** — it disqualifies every v2 number quoted as confirmatory — while the registered primary (rarefaction 38.31 / 46.78 / 13.00 at $m = 84$, $JT = 83.50$ vs null 53.92, $p = 0.0081$) was omitted **for us** — the best-powered registered result in the section, and it had been left out. That the suppressed material included the strongest number as well as the worst label is the reason to state the correction rather than absorb it quietly: selective reporting is not only the omission of unfavourable results, and a reader cannot check which kind occurred unless both are named.

A.2 The reconstructed spread statistic

The v1 preregistration names its spread quantity by the generating kernel’s field, `mean_pairwise_ned`. That kernel is not in the released corpus. What survives is a single logged value in `provenance.json` — `text_dispersion_mean_pairwise_ned = -0.3558` for `Q3_K_M` — negative, so the kernel’s quantity is not a normalized edit distance in the usual sense and its definition is unrecoverable. We substituted an explicit definition: Levenshtein distance over `raw_text` divided by the longer of the two lengths, averaged over unordered within-cell pairs, which on that same cell returns **+0.1060**. The substitution is forced rather than chosen — the preregistration wants the statistic per parent and the kernel logged it only per rung, so no replay was available at the registered granularity even had the definition survived. Section 3’s $p = 0.030$ is the registered test run on a reconstructed quantity; a reader who declines the reconstruction should treat that row as unavailable rather than as evidence in either direction.

A.3 The registered primary echo measure, its degeneracy, and the tie rule

Section 2 of Amendment 1 designates median per-circle centre displacement **the primary echo measure**, specified with a Jonckheere-Terpstra test. Earlier drafts printed the medians as a descriptive “graded companion” and never ran that test — the registration is ambiguous against itself (****Graded companion:**** median per-circle center displacement per sample -- this is the primary echo measure, tested with Jonckheere-Terpstra), inviting the drafts’ reading, but the sentence under the heading is explicit. Run in `sec3_registered_echo_test.py`: `Q4_K_M` ($n=65$) median 0.000000, mean 0.002323, 64/65 rows at exactly zero; `Q3_K_M` ($n=51$) median 0.000000, mean 0.020092, 47/51 at zero; `Q2_K` ($n=49$) median 0.000000, mean 0.002126, 48/49 at zero. **$JT = 4460.0$** against a null mean of 4498.54 (sd 108.41), permutation **$p = 0.686$** (0.6864 unrounded) over 10,000 shuffles. This outcome is unfavourable and had been left unreported while the favourable spread test was reported beside it. It is also degenerate, in the same way and for the same reason as the Amendment 1 rarefaction: 159 of 165 valid rows have a median displacement of exactly zero, so the statistic is very nearly all tie credit and $p = 0.686$ measures saturation rather than flatness — both degeneracies trace to the single-lattice design, whose descriptors were built to detect movement the probe never gave the model an occasion to make. One convention is ours and not registered: the v1 preregistration names Jonckheere-Terpstra but specifies no tie rule, and with 159 of 165 values tied the convention is nearly the whole statistic — we adopt the 0.5 tie credit that wave 2’s addendum states, for consistency across the two waves, flagged as a choice. The registered second half of the quality fork — mean `score_delta` against the parent with bootstrap 95% CIs — was likewise unreported and is now in §3’s table; only `Q4_K_M` excludes zero, and the direction is a small *improvement*, not a decline.

The registered replacement rule, adjudicated in full. Under the rule this returns no category: SURVIVES needs both spread measures at $p < 0.05$ and rarefaction is 0.953; PARTIAL needs monotone score decline and it does not; FALSIFIED needs both spread measures flat and NED is 0.030; UNDERPOWERED does not apply. The honest label is unclassified — this disqualifies the *registered mechanism verdict* specifically, not the probe’s descriptive counts, which §3 reports as descriptive and post-hoc. Two further caveats cut against reading the favourable half alone: the NED decline is **not monotone** (Q3_K_M’s 0.1060 sits above Q4_K_M’s 0.0872, and the entire effect is Q2_K collapsing), and the rarefaction instrument had no resolving power — 159 of 165 valid rows sit at exactly zero displacement, the pooled quantile edges collapse to two occupied cells out of 64, both occupied by every rung, so its $p = 0.953$ is saturation, not evidence of flatness. The prespecified orthogonality check passes its registered criterion (Spearman rho = +0.1672 for the spread descriptor, -0.1675 for the displacement descriptor against score, both far below the registered $|\text{rho}| > 0.5$ confounding threshold), but passing that threshold is not independence: at $n = 165$ both correlations are nominally non-zero ($p = 0.032$ and 0.032). The registered rule asks only whether either descriptor is *confounded* by scoring, and neither is by its stated criterion; a weak residual association remains and is stated rather than rounded away.

A.4 Every p -value reported in §3 and §3.6, its family, and the defect that family carries

Section 3 states the summary of this appendix and points here; nothing below is new evidence.

Nine are two-sided Fisher exact tests on counts ($p = 5.7 \times 10^{-10}, 1.7 \times 10^{-7}, 3.4 \times 10^{-6}, 0.001, 0.007, 0.017, 0.056, 0.44, 1.0$). **Six are permutation tests belonging to the two dispersion probes’ preregistrations** — wave 1’s spread JT (0.030), echo JT (0.686), quality fork (0.454) and rarefaction trend (0.953), and wave 2’s rarefaction JT (0.0081) and score fork (0.3526). **Two are Spearman correlations** run as wave 1’s orthogonality diagnostic (0.032 and 0.032), testing no claim of ours and existing only to check whether a descriptor is confounded by score; wave 1’s preregistration prespecifies the $|\text{rho}|$ criterion but not a p -value threshold, wave 2’s contains no orthogonality clause at all, and both tails are reported for completeness, not as registered tests. Within the Fisher family, the candidate-level contrasts ($5.7 \times 10^{-10}, 0.007, 1.7 \times 10^{-7}, 3.4 \times 10^{-6}, 0.017$) treat rows as independent when they are nested within lineages sharing five seeds across rungs, overstating the evidence — the antecedent study’s appendix declares them not part of any claim, and that status is adopted here for all of them. The seed-level tests (0.001, 0.44) do not have that defect but run on five seeds. No multiplicity correction is applied to the Fisher family and none should be read as implied: across its nine tests a Bonferroni threshold is $0.05/9 = 0.0056$, which neither $p = 0.017$ nor 0.007 meets. The six permutation tests get no correction of their own because each is a named primary or fork inside a locked decision rule, and both rules returned unusable verdicts (unclassified wave 1, FAILED wave 2), so no claim rests on any of the six whatever threshold applies. The two Spearman diagnostics are likewise uncorrected and support no claim.

Twelve are exact lineage-level permutation tests on the post-hoc search-progress statistic — six on accepted improvements (0.0008, 0.0079, 0.0079, 0.135, 0.116, 0.238) and six on final best score (0.593, 0.937, 0.421, 0.318, 0.158, 0.119). This family is the newest and least protected: specified after the data existed, largely the echo contrast in complement, no registration. A Bonferroni threshold across its twelve tests is $0.05/12 = 0.0042$, met only by $p = 0.0008$; neither fresh-seed nor within-ladder tail at 0.0079 survives it, and both are additionally *at the floor* of a five-versus-five enumeration ($2/252$), so no design at this lineage count could have cleared the threshold. These twelve avoid the calls-within-lineage nesting defect (the permutation unit is the lineage), but two weaknesses remain: five lineages per cell is a small pool bounding how small any tail can be, and **the design is seed-crossed while the test is not stratified** — the same five seeds appear at every rung, so the twenty lineages of the pooled comparison are five matched quadruples rather than twenty exchangeable draws, and the enumeration permutes them as though they were. Ignoring positive pairing is conservative for a difference of means, bounding the tails rather than reversing them; the matched alternative (a stratified test permuting rungs within each seed) has a minimum attainable p of $1/16$ on five pairs and could not have produced any of the values above. A call-level Fisher tail on the IQ2 control’s step counts was additionally computed ($p = 0.028$) and is recorded here only to note it is not used: calls within a lineage are not exchangeable, the lineage-level tail on the same counts is 0.135, and the more favourable number is the wrong one.

B fixed-parent control protocol and full registered results

This appendix carries the full protocol, registration text, and complete result tables for the fixed-parent dispersion probe summarized in §3.5.

Design. The probe runs Qwen2.5-Coder-14B at the same three rungs as the main ladder (Q8_0, Q4_K_M, Q3_K_M, Q2_K — Q8_0 registered but never attempted, per "q8_0": {"skipped": "needs 2 gpus, have 1"}) in both waves' `provenance.json`, the single P100 allocated not fitting the 8-bit file), holding the parent constant at each of six preset scores — 0.88, 0.90, 1.04, 1.30, 1.55, 1.65 — and logging coordinates and an echo flag per row. Wave 1 (`sec3_artifacts/dispersion_probe/probe_samples.jsonl`): 288 rows, 165 valid, 123 invalid, of which 18 are harness failures rather than model failures (`gen_error: ValueError: logprobs is not supported`, exactly six per rung — these neither bias between rungs nor belong in a denominator of model behaviour). Wave 2 (`sec3_artifacts/dispersion_probe_v2/`, 432 rows on out-of-sample salted seeds) has none: its rows carry no `gen_error` field, and of its 158 invalid rows exactly one records a genuine model parse failure (`parse_error: bad_triple`), which stays in the denominator.

Registration. The probe carries an analysis preregistration (`analysis_prereg.md`, written before any metric was read, with an amendment timestamped before any output file was downloaded). Wave 2 carries its own preregistration (`analysis_prereg_v2.md`) plus a truncated-run addendum, and its own registered verdict: **FAILED**, because addendum rule R2 requires a ceiling rung (q8_0) to land and it did not, so no trend test is reportable and every wave-2 descriptive is exploratory — the released analyser prints this unprompted (`VERDICT: FAILED / addendum R2: ceiling rung q8_0 did not land; no trend test reportable`).

The v2 registered primary. The pooled rarefied distinct-solution count at matched depth, with a per-parent Jonckheere-Terpstra over 10,000 shuffles. At $m = 84$: $38.31 \pm 1.64 / 46.78 \pm 1.30 / 13.00 \pm 0.00$ distinct solutions across Q4_K_M / Q3_K_M / Q2_K, JT = 83.50 against a null mean of 53.92, permutation $p = 0.0081$. The registered score fork returns JT $p = 0.3526$ — no monotone decline in quality, so this is a variation effect, not general degradation. Two qualifications: the wave is **FAILED** by its own rule, so this number is exploratory however favourable it looks; and it is non-monotone in exactly the way v1's spread test is (46.78 at Q3_K_M sits above 38.31 at Q4_K_M — the whole effect is Q2_K collapsing to 13.00). The replicable statement across both waves is “distinct solutions collapse at the 2-bit rung,” never “distinct solutions decline with quantization.”

The three registered echo-family measures, both waves (`sec3_dispersion_registered.py`):

Table 3: The three registered echo-family measures, both dispersion-probe waves.

measure	Q4_K_M	Q3_K_M	Q2_K
registered centers-only echo (max index-wise centre displacement $< 1e-3$)	64/65 (98%)	47/51 (92%)	48/49 (98%)
registered primary echo measure (median per-circle centre displacement)	0.000000	0.000000	0.000000
retained legacy echo ($1e-5$, all three fields)	34/65 (52%)	31/51 (61%)	46/49 (94%)
legacy echo, v2 replication	56/99 (57%)	41/91 (45%)	77/84 (92%)

All six of the probe's “parents” share identical centres, differing only in radii from a uniform 0.900 grid up to a 1.65 packing with 26 distinct radii, so centre-echo measures whether the model reproduces *that* lattice and cannot observe what would happen with a proposer that moved to a different one. The 92–98% centre-echo figure is a statement about the only lattice ever presented, not a general finding about constructive novelty.

Radius-variation decomposition (post-hoc, constructed after the registered centre measures came back flat, labelled as such everywhere used):

The middle column is identically the legacy echo count in all six cells, so the decomposition adds no evidence there; what it adds is a partition of the non-echo rows into radius-varying and centre-varying — at Q2_K on wave 1, three rows split 2 and 1. Reading down the first column and across waves: the 2-bit endpoint replicates (4% and 6% independently) and the ordering above it does not (46% above 31% on v1, 39% below

Table 4: Radius-variation decomposition by rung (post-hoc, constructed after the registered centre measures came back flat).

rung	centres copied and radii varied	centres copied and radii copied	centres varied
Q4_K_M, v1	30/65 (46%)	34/65 (52%)	1/65
Q3_K_M, v1	16/51 (31%)	31/51 (61%)	4/51
Q2_K, v1	2/49 (4%)	46/49 (94%)	1/49
Q4_K_M, v2	39/99 (39%)	56/99 (57%)	4/99
Q3_K_M, v2	46/91 (51%)	41/91 (45%)	4/91
Q2_K, v2	5/84 (6%)	77/84 (92%)	2/84

51% on v2 — an earlier draft’s “the upper rungs” quoting v1’s 46% alone overstated a single rung’s rate as a pair’s).

All registered outcomes, wave 1, including those against us. One disclosure precedes the table: the spread statistic is reconstructed, not replayed (Appendix A.2), so row 1’s $p = 0.030$ is the registered test on a reconstruction, and a reader who rejects the reconstruction should treat it as unavailable.

Table 5: All registered wave-1 outcomes, including those unfavourable to the hypothesis.

registered analysis	result	direction
primary spread: JT on per-parent mean pairwise NED, valid only, 10,000 permutations	mean NED 0.0872 / 0.1060 / 0.0291; JT = 31.0, $p = 0.030$	declines — favourable
primary echo measure: JT on per-sample median centre displacement, 10,000 permutations	medians all 0.000000; JT = 4460.0 vs null mean 4498.5 (sd 108.4), $p = 0.686$	flat — unfavourable, degenerate
Amendment 1 rarefaction: unique cells on the centres-only descriptor at matched $m = 49$	1.774 ± 0.418 / 2.000 ± 0.000 / 2.000 ± 0.000 ; $p = 0.953$	wrong sign — unfavourable
quality fork: JT on mean score among valid	1.23855 / 1.22889 / 1.25966 ; $p = 0.454$	no monotone decline
quality fork, registered second half: mean <code>score_delta</code> vs parent, bootstrap 95% CI	$+0.00725$ [$+0.0018$, $+0.0139$] -0.00329 [-0.0254 , $+0.0113$] -0.00199 [-0.0211 , $+0.0151$]	/ flat; only Q4_K_M excludes zero /
power floor (rule 4)	$m = 49$, six defined parents per rung	not underpowered

Under the registered replacement rule this returns **no category** (full adjudication in Appendix A.3). It disqualifies the registered mechanism verdict, not the descriptive counts.

Loop-versus-fixed-parent comparison (Q8_0’s condition failed, so this three-rung trend has correspondingly less power than preregistered):

Table 6: Loop-versus-fixed-parent echo comparison by rung.

rung	echo, fixed parent (all)	echo, parent 0.88-0.90	echo, loop
Q4_K_M	34/65 (52%)	6/18 (33%)	3/20 (15%)
Q3_K_M	31/51 (61%)	7/16 (44%)	3/19 (16%)
Q2_K	46/49 (94%)	11/12 (92%)	17/18 (94%)

At the 2-bit rung the designs agree. At the upper rungs they do not: held to a fixed parent, Q4_K_M and Q3_K_M echo far more often than the loop suggests. Restricted to the 0.88-0.90 band the loop actually occupies, the contrast is **6/18 versus 11/12 — 33% against 92%**, not 15% against 94%.

Per-parent vectors, in full:

Table 7: Per-parent echo vectors, in full.

parent	0.88	0.90	1.04	1.30	1.55	1.65
Q4_K_M	4/12 (33%)	2/6 (33%)	9/14 (64%)	4/11 (36%)	9/11 (82%)	6/11 (55%)
Q3_K_M	5/10 (50%)	2/6 (33%)	4/7 (57%)	5/11 (45%)	9/10 (90%)	6/7 (86%)
Q2_K	7/8 (88%)	4/4 (100%)	8/8 (100%)	12/12 (100%)	8/8 (100%)	7/9 (78%)

Consequences. The cliff’s direction and 2-bit endpoint survive; its spread does not — the loop’s “14% to 94%” is partly an artifact of where each rung’s running parent sits, and the fresh-seed 79%-versus-6% figure and the invalid-row near-copy gradient carry the same confound. The §3.4 IQ2 scheme-gradient ladder (6% $\rightarrow$ 43% $\rightarrow$ 75-79%) is weakened: if Q4_K_M’s matched rate is 33-52% rather than 6%, IQ2_M’s 43% may not be separated from Q4_K_M at all, and the gradient is withdrawn as evidence about bit width, kept only as a description of loop-condition numbers. The registered upper-rung bound “ $\leq 35\%$ ” held under the loop condition (11-16%) and would have **failed** under fixed parents (52-61%) — correctly scored against the loop design it was registered for, but not a general property of those rungs. The probe is not a substitute for the loop measurement — a fixed parent removes the lineage history the loop includes — but is the better-controlled comparison on the one axis where the loop is confounded.

C lineage-level search-progress records

This appendix carries the full per-lineage records and the horizon-power exercise summarized in §3.6.

Per-lineage accepted-step records, registered 14B ladder (five lineages of ten generations each; an accepted improvement is a valid output scoring strictly above the lineage’s running best):

Table 8: Per-lineage accepted-step records, registered 14B ladder.

rung	accepted steps per lineage	total	valid	echo	final best per lineage
Q2_K	0, 0, 0, 0, 1	1/50	18	17	0.900, 0.900, 0.900, 0.900, 1.625
Q3_K_M	4, 3, 4, 2, 2	15/50	19	3	1.040, 1.300, 0.962, 1.625, 1.248
Q4_K_M	6, 3, 2, 3, 2	16/50	20	3	1.060, 1.300, 0.936, 1.040, 0.933
Q8_0	2, 2, 5, 1, 4	14/50	18	2	0.936, 1.300, 0.926, 1.083, 1.040

Four of five Q2_K lineages take zero steps; their running parent is still the seeded 6×5 grid at generation nine. Because an echo can never be an accepted improvement (**accepted** $\leq$ **valid** - **echo**), the identity gap k between the two — 0, 1, 1, 2 across the four rungs — is the count of rows that departed and still failed to beat the parent.

Conditional-quality per-parent breakdown (pooled across both dispersion-probe waves, post-hoc, `sec3_conditional_quality.py`): Q4_K_M 164 valid, 74 departures, 60/74 (81%) improved, 95% CI [70%, 89%]; Q3_K_M 142 valid, 70 departures, 61/70 (87%), CI [77%, 94%]; Q2_K 133 valid, 10 departures, **8/10 (80%)**, CI [44%, 97%]. Fisher on 8/10 vs 60/74, $p = 1.00$. Per-parent, where Q2_K has data: 0.880, 2/2 against Q4_K_M’s 21/23 (91%); 0.900, 1/2 against 10/11; 1.040, 2/2 against 8/9 (89%); 1.650, 3/4 against Q4_K_M’s 6/9 (67%) and Q3_K_M’s 4/6. Four of Q2_K’s ten departures sit at the hardest parent (1.650), 40% of its sample against 12% of Q4_K_M’s — its departures are not drawn from easier parents. Scored against a threshold we wrote ourselves the same day, in an unlocked note (`wave3_prereg_heilbronn.md`, explicitly not a registration): the power floor (25 departures required, 10 observed) is not met, so no branch is confirmed. Which reference is chosen moves the exclusion line — half of Q4_K_M’s 81% is 40.5%, half of the pooled upper rungs’ 84% is 42.0%, half of Q3_K_M’s (the better-performing comparator) 87% is 43.6%. The 95% Clopper-Pearson lower bound on 8/10 is **44.4%**, excluding the collapse branch against all three by 3.8, 2.4 and **0.8** points respectively; had one of the ten departures failed to improve, 7/10 gives a lower bound of 34.8% and the exclusion fails against every comparator.

Replication tests, exact lineage-level permutation (distinct from the dispersion probes’ JT tests — shares only the word “permutation”): registered ladder $p = 0.0008$ against the pooled upper three (15,504 splits), $p = 0.0079$ against Q4_K_M alone (252 splits — the floor of a five-versus-five enumeration, the smallest two-sided tail such a design can return whatever the effect size). Fresh seeds: 3 steps against 14, $p = 0.0079$ (also floor-bound). IQ2 control: imatrix Q2_K 6 steps against IQ2_M’s 16, $p = 0.135$ — does not replicate. A call-level Fisher test on the same counts returns $p = 0.028$; the lineage-level figure is reported instead because calls within a lineage are not exchangeable (improvement at call k depends on what landed earlier in the same lineage), which costs the IQ2 replication but avoids banking an unsupported significance.

A design capable of distinguishing a large effect from an enormous one needs more lineages per rung, not more generations per lineage — eight per rung would lower the floor to 1.6×10^{-4} .

Outcome-level (final best score) tails, all non-rejection at five lineages per cell: registered ladder, Q2_K mean 1.045 vs pooled 1.115 ($p = 0.593$), vs Q4_K_M 1.054 ($p = 0.937$); fresh seeds, Q2_K 1.153 vs 0.999 ($p = 0.421$); IQ2 control, 1.087 vs 0.922 ($p = 0.318$); 7B, $p = 0.158$ against pooled others, 0.119 against Q4_K_M — six tails in all, $p = 0.12$ – 0.94 , behind the abstract’s statement that final best score separates the rungs nowhere.

The horizon-power exercise, run and mostly withdrawn. `sec3_horizon_power.py` resamples the observed process (departure rate and empirical offer distribution per rung from the pooled 14B ledgers, a lineage of T calls drawing $\text{Binom}(T, d)$ offers, best-so-far the max of those and the seed). It cannot support a power statement about the paper’s own design: the only alternative the model supplies is its own projection, which puts Q2_K *ahead* at every horizon, because the 2-bit rung’s six observed departure-offers are 0.867, 0.900, 1.040, 1.300, 1.625, 1.625 — the maximum appears twice and one entry is the seed score itself, handing the resampler a one-in-three jackpot and a hard ceiling. Six observations cannot carry a direction, so they cannot carry an effect size either; an earlier draft’s power claim is withdrawn, and the outcome-level null stands as non-rejection with no quantitative statement about how underpowered it is. It also cannot support “widen rather than lengthen”: cost-equalised, lengthening wins (five lineages at 100 generations is 500 calls for 84%; twelve at 50 is 600 calls for 82%), and at $T = 10$ the simulated rejection rate *falls* as lineages are added (17%, 8%, 5%, 4% for 5, 8, 12, 20 lineages) — not how power behaves, a sign the coarse five-versus-five permutation tail is doing the work rather than any effect. What survives is structural: final best score is a maximum, insensitive to draw count by construction — multiplying effective draws moves it by the distance between two upper quantiles of the same offer distribution, not proportionally to the draw ratio. A wave demonstrating harm should use time-to-threshold, area under the best-so-far curve, or the accepted-step count itself (which this appendix measures and which the wave-3 design note names as its primary), not final best score. The horizon run remains an open gap, not closed by arithmetic.

D opus_alias forensic detail

This appendix carries the narrative and secondary-comparator detail summarized in §4.

Reported-token withdrawal, in full. Reported-token counts were uniform at $\sim 49.9\text{k}$ across all thirty invocations, tight enough to read as a second serving signature. It does not survive definition. The harness log records a single per-invocation usage figure not disaggregated into input, output and cache-read; the thirty stored completions are 215–1283 characters (mean 578), roughly 61–367 output tokens, so $\sim 49.9\text{k}$ cannot be an output count. The reading it supports is a total/context counter over a large fixed system prompt, inherited user-level instruction files, and a task prompt byte-identical in length across all three cells (520 characters, the template substituting a two-digit N). Under that reading near-uniformity is exactly what the design predicts, and the residual spread is plausibly tokenizer-level variation between “13”, “21” and “31”. We report it as a non-finding because the reasoning generalizes to any study reading a single aggregate usage integer.

Throughput does not order by bit-width, in detail. Across §3’s ladders: on the 14B ladder Q8_0 is the slowest of four rungs at 14.9 tok/s against Q2_K’s 22.5; on the 7B ladder FP16 is slower than every quantized rung, and Q3_K_M sits below Q4_K_M at both scales (`sec6_cv_canary_audit.py`). Those gaps are tens of percent and this arm’s gap is one to two orders, so the magnitudes are not comparable and this does not overturn the reading that fast completions favour H1 — but it does mean speed and precision are separate axes even on a fixed stack. What the ladders further show (§6 item 4) is that throughput separates served files within a single wave on fixed hardware once warm-up and degenerate outputs are excluded, but not across waves and not across hardware, where the ordering between two SHA-pinned files reverses — not a portable fingerprint, and not usable to attest a serving path by anyone who does not control the machine.

The registered scorecard, full detail. The working log marked P-O1, P-O2 and P-O4 NOT EVALUABLE, reasoning that scoring a tier comparison across a validity collapse would be dishonest. That verdict is too conservative for P-O1, which is arithmetically evaluable and **held**: registered that the on-trap rate pooled

across cells would be at most Sonnet’s 1/30, observed **0/30** — no valid row falls within the registered **2e-3** window of 1.625, 2.1 or 2.5833, nearest miss 2.07 against a 2.1 trap, off by 0.03, fifteen times the window. P-O3 (multi-radii fraction ≥ 0.9 of valid samples) **held**, 4/4, though on four rows it carries little weight. P-O2 (rival-argmax) and P-O4 (best valid score ≥ 2.2588835) **failed outright**: zero rival-argmax outputs against Sonnet’s 6/30, best valid 2.07. Two held, two failed, none unevaluable — weaker than the registration hoped on P-O2/P-O4, stronger than “not evaluable” on P-O1.

What the observable set separates, in detail. The runtime exposes, per invocation, exactly four things: completion text, wall-clock duration, a single aggregate usage figure, and error codes (set O). On the text channel the two hypotheses induce the same distribution — both predict ambitious families executed with broken tangencies — so no amount of text observation separates them. On the timing channel they are not symmetric: H1 predicts the latency observation directly, H2 is silent on it, and to cover O H2 must be conjoined with an auxiliary (“a fast serving path was in use but had no behavioral effect”) that is H1’s mechanism minus its causal claim, strictly more complex. More sampling does not separate the hypotheses — the anomaly is uniform, so more of it is more of the same — and timing instrumentation does not either, because the serving path is not a variable settable from inside the runtime.

The pinned-endpoint route, not run. The same thirty hash-pinned prompts could be issued to a dated model identifier on a pinned inference endpoint, entirely outside the agent runtime: if the endpoint reproduces the ambitious-family, broken-tangency signature at comparable validity, H2 gains substantially; if it returns 30/30-class competence, H1 does. What such a run still cannot do is attest what the *alias* resolved to on the day sampled — the counterfactual is gone, which is the sense in which C3 survives regardless. We did not run this experiment; its absence is a limitation of this paper rather than a property of the problem.

Prompt-variation and re-snapshot designs, as originally specified. An earlier dismissal — that prompt variation cannot help because both hypotheses act on execution rather than intent — is wrong for at least one design: hand the alias a fixed, known-valid packing and ask it to verify or repair the tangencies. The model did not choose that construction, so arithmetic execution is decoupled from constructive ambition, an H1/H2 discriminator. Re-sampling the same alias at a later date is informative either way and runnable from inside the runtime, narrowed to: a re-snapshot can detect a *change* in the serving signature without ever attesting the serving path in either snapshot. Both designs were preregistered together in `arm_r_preregistration.md` and run on 2026-08-07 (§4.1), where B2 disconfirmed the anchor prediction P-R0 and the repair probe (arm R) ran under a decision rule that forbids drawing an H1/H2 conclusion from it once the anchor changed.